Trojan-Resilient Reverse-Firewall for Cryptographic Applications

Chandan Kumar*[1], Nimish Mishra[1], Suvradip Chakraborty[2], Satrajit Ghosh[1], and Debdeep Mukhopadhyay[1]

[1]IIT Kharagpur
[2]VISA Research USA

Abstract

Reverse firewalls (RFs), introduced by Mironov and Stephens Davidowitz at Eurocrypt 2015, provide a defence mechanism for cryptographic protocols against subversion attacks. In a subversion setting, an adversary compromises the machines of honest parties, enabling the leakage of their secrets through the protocol transcript. Previous research in this area has established robust guarantees, including resistance against data exfiltration for an RF. In this work, we present a new perspective focused on the implementation specifics of RFs. The inherently untrusted nature of RFs exposes their real-world implementations to the risk of Trojan insertion — an especially pressing issue in today's outsourced supply chain ecosystem. We argue how Trojan-affected RF implementations can compromise their core exfiltration resistance property, leading to a complete breakdown of the RF's security guarantees.

Building on this perspective, we propose an enhanced definition for "Trojan-resilient Reverse Firewalls" (`Tr-RF`), incorporating an additional Trojan resilience property. We then present concrete instantiations of `Tr-RF`s for Coin Tossing (CT) and Oblivious Transfer (OT) protocols, utilizing techniques from Private Circuit III (CCS'16) to convert legacy RFs into `Tr-RF`s. We also give simulation-based proofs to claim the enhanced security guarantees of our `Tr-RF` instantiations. Additionally, we offer concrete implementations of our `Tr-RF` based CT and OT protocols utilizing the Open-Portable Trusted Execution Environment (OP-TEE). Through OP-TEE, we practically realize assumptions made in Private Circuit III that are critical to ensuring `Tr-RF` security, bridging the gap between theoretical models and real-world applications. To the best of our knowledge, this provides the *first* practical implementation of reverse firewalls for any cryptographic functionality. Our work emphasizes the importance of evaluating protocol specifications within *implementation-specific* contexts.

*Partially supported by Google Asia Pacific Pte Ltd. as part of the Google PhD fellowship program.

Contents

1 Introduction

The concept of **Reverse Firewall (RF)** was introduced by Ilya Mironov and Noah Stephens-Davidowitz in [MSD15], addressing a pivotal question: *"Can we design cryptographic protocols that achieve meaningful security when the adversary may arbitrarily tamper with the victim's computer?"*

This concern became especially relevant in the post-Snowden era, which exposed that user hardware and software could be compromised to leak sensitive information, such as through backdoors, often without the user's knowledge [MSD15]. Introduced in [MSD15] and followed up in subsequent works [DMSD16, CDN20, CGPS21, CGS23, CMNV22, BBF$^+$20, CMMV25], Reverse Firewalls (RF) serve as a solution to this problem. An RF acts as a firewall, between the user's machine and the external world, sanitizing messages sent and received during cryptographic protocols in a functionality-preserving way. Informally, it achieves the following key objectives:

- *Maintains functionality:* Assuming the user's machine is operating correctly, the RF should not alter the functionality of the protocol.
- *Preserves security:* If a cryptographic protocol ensures security based on the assumption that the user's protocol implementation is not tampered with, the RF maintains the same level of security, irrespective of the behavior of the user's computer.
- *Prevents exfiltration:* If the user's machine is compromised, the RF prevents it from leaking sensitive information to the external environment.

Trust Assumptions on RFs: Note that, in this settings the RF is not considered a trusted third party and does not have access to the user's internal secrets or private states. Its access is strictly limited to the public parameters, the incoming and the outgoing messages of the protocol. The RF's primary role is to modify the protocol transcript in a manner that ensures no adversary, even one with control over the user's machine, can exfiltrate information or embed covert signals within the protocol transcripts via the RF. Consequently, in subversion settings, the security of a cryptographic protocol relies critically on the correct implementation of the RFs. More specifically, if an RF fails to modify the protocol transcript as prescribed, it can lead to information leakage, defeating the very purpose of using RFs. To further motivate this, let us consider the case of a simple two-party coin tossing protocol, where Alice and Bob engage engage in a protocol to agree on a random bit. Alice initially commits to a bit by sending a commitment com to Bob. Now if Alice's implementation is tampered, it may maliciously choose the commitment string com (say by choosing a bad randomness to commit or reverse sampling the commitment string until the commitment string starts with a fixed number of zeros) to covertly exfiltrate some signal or some information about Alice's internal secret state. In an RF setting, a correctly implemented RF randomizes com and mitigates the possibility of such leakage. Thus, to provide security against such a malicious implementation, we implicitly assume that the RF will randomise the protocol transcript according to the specification. However, a subverted implementation of RF might not follow the protocol description and may not randomise com in the prescribed manner. That will break all the security guarantees of such a system. Furthermore, in today's outsourced supply chain ecosystem, malicious actors can embed Trojans [WS10, WTP08] within RF implementations. Such Trojan-affected RF implementations can compromise their exfiltration resistance property, leading to a complete breakdown of the RF's security guarantees.

One might argue that composing multiple reverse firewalls $\{RF_1, \cdots, RF_m\}$, to form a single secure reverse firewall could address this issue, assuming that at least one RF_i maintains functional correctness and adheres to the protocol description—a property referred to as *stackability* in [MSD15]. However, this approach still relies on the existence of at least one functionally correct and secure RF, which is challenging to ensure in practice. In this paper, we address this issue by considering the following question:

Is it possible to propose a generic and practical implementation framework that facilitates secure implementation of a reverse firewall?

We affirmatively answer this question and present the first practical implementation of reverse firewalls for two cryptographic functionalities: coin tossing and oblivious transfer. Our approach leverages a *split*

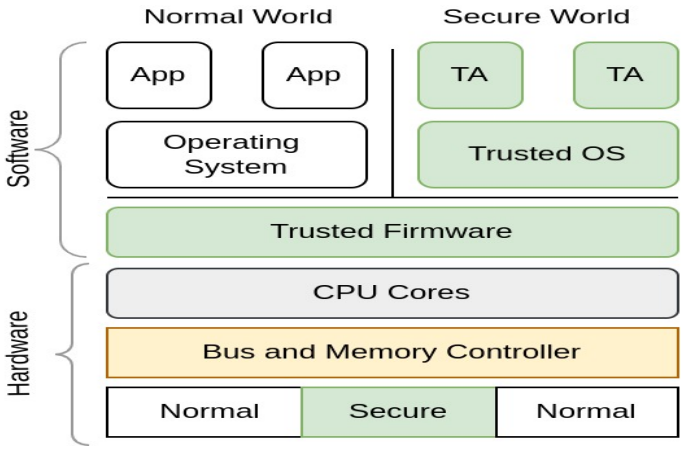

Figure 1: High-level architectural view of ARM TrustZone

manufacturing framework [DFS16] for implementing reverse firewalls, where the RF circuit is divided into multiple sub-circuits connected through a simple master circuit. The master circuit, implemented in-house, serves as the root of trust. This framework ensures that even if the sub-circuits are tampered with, a correct implementation of the master circuit guarantees the functional correctness of the RF. We require the master circuit to be as *simple* as possible, comprising a minimal number of wires and basic gates. Importantly, the size of the master circuit remains independent of the complexity of the RF specification. Further, we define the concept of trojan-resilient RF (`Tr-RF`) and prove that one can take any RF specification and make it trojan-resilient using the proposed framework.

1.1 Our Contributions

Trojan resilient reverse firewall We introduce the concept of a Trojan-Resilient Reverse Firewall (Tr-RF). In Section 3, we formally define the key properties required for a Tr-RF: *robust functionality maintenance*, *robust security preservation*, and *robust exfiltration resistance*. These properties generalize and strengthen the standard security requirements of a reverse firewall, namely functionality maintenance, security preservation, and exfiltration resistance. We show that any standard exfiltration-resistant RF [MSD15] can be combined with a robust Trojan protection scheme (Definition 4) to obtain a Tr-RF that achieves robust exfiltration resistance.

An implementation framework for *Tr-RF* We propose a framework for transforming any legacy RF into a Tr-RF using the robust trojan protection scheme, Private Circuit III [DFS16]. This framework decomposes the RF specification into sub-circuit specifications, which are interconnected through a master circuit M. By designating M as the root of trust, the sub-circuits can be securely combined to construct a Tr-RF. The design of M is kept as simple as possible, as it is expected to be developed in-house, while the implementation of more complex sub-circuits can be outsourced to external agents. With a trusted implementation of M, the resulting Tr-RF remains secure even if the sub-circuits are compromised.

Concrete instantiation and implementation We have instantiated and implemented Tr-RF for two cryptographic protocols: coin tossing (CT) [Blu81] and oblivious transfer (OT) [NP01, AIR01]. To the best of our knowledge, this provides the *first* practical implementation of reverse firewalls for any cryptographic functionality.

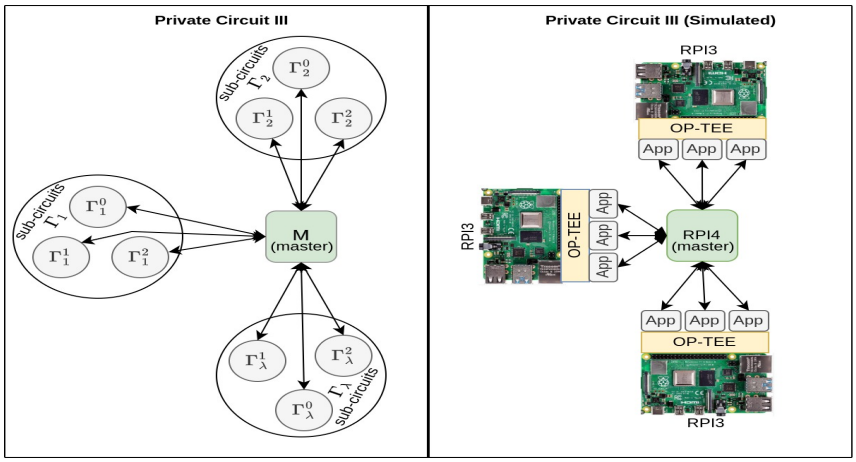

Figure 2: Trojan-resilient circuit architecture and its simulation using the Raspberry Pi OP-TEE framework, where trusted components are shown in green and untrusted components in gray.

To instantiate coin-tossing, we start with Blum's protocol [Blu81] and propose an RF construction based on it. We prove that this construction is a secure RF and, with a robust implementation, it results in a Tr-RF for the coin-tossing functionality. For OT, we consider the RF OT protocol from [MSD15] and compile it to create a Tr-RF OT construction. Finally, we present a practical implementation of both Tr-RF CT and Tr-RF OT in the Open-Portable Trusted Execution Environment.

We implement the framework for `Tr-RF`s by leveraging the Open Portable Trusted Execution Environment (OP-TEE) on ARM TrustZone (see Figure 2). *We emphasize that we do not use TEEs to ensure functional correctness of RFs.* TEEs are used solely to realize certain specific assumptions of Private Circuit III by reducing them to collision-resistance of a cryptographic hash function. The functional correctness of `Tr-RF`s is guaranteed by Private Circuit III, along with some additional modifications that we introduce and discuss henceforth.

In this work, we implement `Tr-RF` in a distributed systems setting over a LAN network, leading to overheads in performance and communication. However, in the setting of split manufacturing, `Tr-RF` will be printed on a *single* Printed Circuit Board (PCB), thereby eliminating these overheads.

2 Preliminaries

Notations. We write $x \xleftarrow{\$} \mathcal{X}$ to represent that an element x is sampled randomly from a set/distribution $\mathcal{X}$. The output x of a deterministic algorithm A is denoted by $x = \mathrm{A}$ and the output x' of a randomized algorithm A$'$ is denoted by $x \leftarrow \mathrm{A}'$. We denote the adversary by symbol $\mathcal{A}$. We refer to $k \in \mathbb{N}$ as the computational security parameter and denote by $\mathsf{negl}(k)$ negligible function in k.

2.1 Commitment Scheme

A commitment scheme allows a party, called the *committer*, to commit to a value while keeping it hidden, with the ability to reveal the committed value later.

Definition 1 (Commitment Scheme)**.** A commitment scheme is defined as a tuples of algorithms $\mathsf{COMMIT} = (\mathsf{Com}, \mathsf{Reveal})$,

- **Com:** The committer commits to message $m \in \mathcal{M}$ using a randomness $r \in \mathcal{R}$ as $\mathsf{com} = \mathsf{Com}(m; r)$.

- **Reveal:** The committer provides the original message m and randomness r to verifier to verify the commitment as, $\mathsf{Com}(m;r) \stackrel{?}{=} \mathsf{com}$.

The commitment scheme must satisfy the following properties:

- **(Hiding).** For a security parameter k, and a stateful adversary $\mathcal{A}$, we have:

$$\Pr\left[b' = b \;\middle|\; \begin{array}{l}(m_0, m_1, \mathsf{st}) \leftarrow \mathcal{A}(1^k);\ b \stackrel{\$}{\leftarrow} \{0,1\};\\ \mathsf{com} \leftarrow \mathsf{Com}(m_b);\ b' \leftarrow \mathcal{A}(\mathsf{com}, \mathsf{st})\end{array}\right] \leq \frac{1}{2} + \mathsf{negl}(k).$$

  A commitment is scheme is *perfectly hiding*, if the above probability is equal to $1/2$.

- **(Binding).** For all non-uniform polynomial time stateful adversaries $\mathcal{A}$, we have,

$$\Pr\left[\begin{array}{c} m \neq m' \wedge \\ m, m' \neq \perp \end{array} \;\middle|\; \begin{array}{l}(m, m', r, r') \leftarrow \mathcal{A}(1^k) : \\ \mathsf{Com}(m;r) = \mathsf{Com}(m';r')\end{array}\right] \leq \mathsf{negl}(k).$$

Homomorphic Commitment Scheme [Gro09, CDN20]: The commitment scheme COMMIT is homomorphic if for all $m, m' \in \mathcal{M}$, $r, r' \in \mathcal{R}$ we have:

$$\mathsf{Com}(m;r) \cdot \mathsf{Com}(m';r') = \mathsf{Com}(m + m'; r + r')$$

In this work, we focus on Pedersen homomorphic commitment scheme [Ped91] which is perfectly hiding and computationally binding.

2.2 Coin Toss Functionality

A coin toss functionality is a cryptographic primitive that allows two or more parties to jointly generate a random bit in a way that ensures fairness and unpredictability.

Definition 2 (Coin Toss Functionality)**.** Let $(P_1, P_2, \ldots, P_n)$ be n parties and k be the security parameter then the coin toss functionality can be defined as,

- **Input:** Each party P_i $(1 \leq i \leq n)$ provides an input x_i, which is typically a random bit.
- **Output:** The protocol outputs a single random bit b (coin value), computed as a function of all the parties' input bits.

The coin toss output b should be uniformly distributed in $\{0, 1\}$.

In this work, we focus on the two party coin tossing protocol from [Blu81].

2.3 Oblivious Transfer Protocol

Oblivious Transfer (OT) is a cryptographic protocol where a sender has two messages, and the receiver can choose to learn one of them without revealing their choice. At the same time, the sender remains oblivious to which message was chosen.

Definition 3 (Oblivious Transfer (OT))**.** An Oblivious Transfer (OT) [NP01] protocol is a two-party cryptographic primitive between a sender S and a receiver R, defined as follows:

- **Inputs:** S holds two messages $m_0, m_1 \in \mathcal{M}$ and R holds a choice bit $b \in \{0, 1\}$.
- **Outputs:** R learns m_b corresponding to their choice bit b. S learns nothing about the choice bit b.

The protocol should ensure that S learns no information about b, and R learns only m_b and gains no information about m_{1-b}.

In this work, we use upon Naor-Pinkas/Aiello-Ishai-Reingold [NP01, AIR01] oblivious transfer and the version of oblivious transfer protocol given in [MSD15].

2.4 Hardware Trojans and Trojan protection schemes.

Trojans [WS10, WTP08] are malicious modifications or hidden components inserted into hardware or software systems to compromise their integrity, functionality, or security. These Trojans remain undetected during normal operation and activate under specific conditions. This work focuses on *digitally triggered Trojans* [WS11], which activate harmful functionality upon receiving specific digital inputs. Triggering mechanisms include "cheat codes," where specific input patterns activate the Trojan, and "time bombs," which trigger malicious behavior after a set duration or number of executions. Often arising from outsourced manufacturing in modern supply chains, these Trojans pose significant threats. Waksman and Sethumadhavan [WS11] proposed scrambling runtime inputs to prevent malicious actions, while [DFS16] introduced a compiler (so called Trojan protection scheme) to transform circuits into trojan-resilient versions. Details on trojan protection scheme can be found in Section 3.1.

2.5 Trusted Execution Environment

Trusted Execution Environment (TEE)[JSS20]. To address software vulnerabilities, including kernel-level issues, the security community has adopted hardware-backed mechanisms such as ARM TrustZone, Intel's SGX (Software Guard Extensions) and TDX (Trusted Domain Extensions), and AMD's PSP (Platform Security Processor). These solutions partition System-on-Chip (SoC) resources into trusted and untrusted execution environments, ensuring that critical operations occur in the trusted environment. This hardware-enforced isolation adds a layer of security, protecting sensitive data even if the untrusted environment is compromised. ARM TrustZone, introduced in ARM chipsets since ARMv6, exemplifies such a solution (see Figure 1). It divides the SoC into the Secure World (Trusted Execution Environment, TEE) and the Normal World (Rich Execution Environment, REE). While applications in the REE (client applications) are vulnerable to bugs, those in the TEE (trusted applications) are isolated to maintain data integrity and confidentiality. Shared resources, such as storage and peripherals, are securely partitioned through hardware mechanisms, ensuring sensitive operations remain protected even if the REE is compromised. For practical implementation of the generic compiler proposed in Private Circuit III, we utilize the ARM TrustZone-based Open Portable Trusted Execution Environment (OP-TEE).

3 Trojan Resilient Reverse Firewall (Tr-RF)

In this section, we present our definitions of trojan resilience for reverse firewalls. Prior to introducing our definitions, we recall the definition of a trojan protection scheme as outlined in [DFS16].

3.1 Trojan Protection schemes [DFS16].

A trojan protection scheme $\Pi = (\mathsf{TR}, \mathsf{T})$ consists of a circuit transformation algorithm TR and a testing algorithm T. The circuit transformation algorithm TR compiles an arbitrary functionality, described as an arithmetic circuit Γ into a protected form that includes a trusted master circuit $\mathcal{M}$ and a set of circuit specifications $\Gamma_1, \ldots, \Gamma_\ell$. Importantly, the master circuit $\mathcal{M}$ contains only a few wires and simple gates, keeping its size independent of the complexity of Γ. This transformation is based on the "Split Manufacturing" principle proposed by Imeson et al. in [IEGT13]. The security of the trojan protection scheme Π against a malicious manufacturer is modeled through a robustness game, denoted as ROB_Π, as shown in Figure 3. In the game, the transformation TR is first run to generate the specification of the protected circuit

$(\mathcal{M}, \{\Gamma_i\}_i, \vec{m}')$. This specification is then provided to the malicious manufacturer $\mathcal{A}$, who produces a set of devices $\{D_i\}_i$. Following [DFS16], we require that an implementation of D_i can be simulated.

Assumption 1. *Let D_i be the devices output by $\mathcal{A}$. We require that there exists (possibly probabilistic) circuit specifications $\widetilde{\Gamma}_i$ such that for all public inputs $\vec{x}$ and any initial state $\vec{m}$, the views of the circuit D_i and $\widetilde{\Gamma}_i$ are identically distributed.*

The second component, the tester T is then run to verify that each device D_i correctly implements the functionality Γ_i, ensuring that its input / output behavior matches that of the honest specification. If the devices pass the tests, the adversary enters the second phase where they interact with a system consisting of the trusted master $\mathcal{M}$ and the device D_i. The evaluation of the sub-circuits $D_1, \ldots, D_\ell$ with master $\mathcal{M}$ on input $\vec{x}$ with initial state $\vec{m}$ to produce output $\vec{z}$ will be written as $\vec{z} \leftarrow (\mathcal{M} \Longleftrightarrow D_1, \cdots, D_\ell)[\vec{m}](\vec{x})$. This composition $(\mathcal{M} \Longleftrightarrow D_1, \cdots, D_\ell)$ can be viewed as a circuit $\mathcal{C}_{\mathcal{W}}$ composed of the sub-circuits Γ_i and $\mathcal{M}$, where the composition is specified by the communication commands between Γ_i and $\mathcal{M}$. We say that an adversary $\mathcal{A}$ wins the game if and only if, after the testing succeeds, they produce an output $\vec{z_i}$ that deviates from the correct output $\vec{y_i}$ computed on input $\vec{x_i}$, i.e., $\vec{y_i} \leftarrow \Gamma[\vec{m}](\vec{x_i})$. In other words, robustness guarantees that for identical inputs, the outputs in the second phase match those generated by the honest specification Γ. Robustness is parameterized by two values: t and n, where t represents the number of tests performed by T, and n is the number of executions in which the device's output align with the honest specification Γ. Following [DFS16], constructing a robust trojan protection scheme requires that $t \gg n$.

Definition 4 (Trojan robustness)**.** Let ℓ, r, t, k be some natural parameters. A trojan protection scheme $\Pi = (\mathsf{TR}, \mathsf{T})$ is (t, r, ϵ)-trojan robust if the following two conditions hold:

- The tester T is t-bounded, i.e., each of the devices D_i is run by T at most t times, and at the end outputs a bit b indicating whether the test has passed or failed.
- For any (potentially malicious) manufacturer $\mathcal{A}$, any circuit Γ and any initial state $\vec{m}$ we have:

$$\Pr[\mathsf{ROB}_\Pi(\mathcal{A}, \mathsf{pub} = (\ell, t, r, k), \Gamma, \vec{m}) = 1] \leq \epsilon,$$

  where the probability is taken over the internal coin tosses of $\mathcal{A}$ and and the coin tosses of the game ROB_Π.

We will need the following theorem from [DFS16].

Theorem 1. Suppose $t, r, \ell, k \in \mathbb{N}$ with k is the computational security parameter, and let $\lambda = \ell/3$. Then $\Pi = (\mathsf{TR}, \mathsf{T})$ is (t, r, ϵ)-trojan robust for $\epsilon := F(\lceil \lambda/2 \rceil; \lambda, r/t) + \mathsf{negl}(k)$. In particular:

- For $r < 4t$ the scheme Π is (t, r, ϵ)-trojan robust for $\epsilon := (4r/t)^{\lceil \lambda/2 \rceil} + \mathsf{negl}(k)$, and
- For $r < 2t$ the scheme Π is (t, r, ϵ)-trojan robust with $\epsilon := e^{-2\lambda(1/2 - r/t)^2} + \mathsf{negl}(k)$.

Here, $F(k; \lambda, p)$ is the tail of a binomial distribution with λ trails, p being the success probability and k being the minimal number of successes, i.e.:

$$F(k; \lambda, p) := \sum_{i=k}^{\lambda} \binom{\lambda}{i} p^i (1-p)^{\lambda - i} \tag{1}$$

In the next section we formally present our definition of trojan-resilient cryptographic reverse firewalls.

3.2 Trojan-Resilience for Cryptographic Reverse Firewalls.

Before presenting our new definitions for trojan-resilient reverse firewalls, we establish some notations and concepts related to cryptographic protocols and parties.

Game $\mathbf{ROB}_{\Pi}(\mathcal{A}, \mathbf{pub} = (\ell, t, r, k), \Gamma, \vec{m})$

$\left((\mathcal{M}, \{\Gamma_i\}_i), \vec{m}'\right) \leftarrow (\mathsf{TR}_1(1^k, \ell, \Gamma), \mathsf{TR}_2(1^k, \ell, \vec{m}))$

$\{D_i\}_i \leftarrow \mathcal{A}(1^k, \mathcal{M}, \{\Gamma_i\}_i)$;

Set the initial state of the devices $\mathsf{Init}(\{D_i\}_i, \vec{m}')$;

If $\mathsf{T}^{D_1(\cdot),\cdots,D_\ell(\cdot)}(1^k, (\mathcal{M}, \{\Gamma_i\}_i)) = \mathit{false}$, return 0;

$\vec{x_1} \leftarrow \mathcal{A}(1^k)$;

For $i = 1, \cdots, r$ repeat:

  $\vec{z_i} \leftarrow (M \Longleftrightarrow D_1, \cdots, D_\ell)[\vec{m}'](\vec{x_i})$;

  $\vec{y_i} \leftarrow \Gamma[\vec{m}](\vec{x_i})$;

  If $y_i \neq z_i$ then return 1;

  $\vec{x_{i+1}} \leftarrow \mathcal{A}(1^k, \vec{z_i})$;

Return 0.

Figure 3: The robustness game for a trojan protection scheme.

Definition 5 (Cryptographic Protocols [MSD15]). A cryptographic protocol $\mathcal{P}$ is defined as an interaction between stateful parties $P_1, \ldots, P_n$. First, a $\mathsf{setup}(1^k)$ procedure returns: - an initial state for each party $(\sigma_{P_i})_{i=1}^n$, referred to as their respective inputs, - a public parameter ρ, - a message schedule[1].

The parties then proceed to exchange messages according to the schedule. Each party uses a next-message algorithm $\mathsf{next}_{P_i}(\sigma_{P_i})$ to determine the message to send and a message-receipt algorithm $\mathsf{receive}_{P_i}(\sigma_{P_i}, m)$ to update its state upon receiving a message. Once the protocol concludes, each party runs its output algorithm $\mathsf{output}_{P_i}(\sigma_{P_i})$ to produce the final result.

The protocol is identified by its parties and setup procedure: $\mathcal{P} = (\mathsf{setup}, (P_i)_{i=1}^n)$. Each party is characterized by its defining algorithms: $P_i = (\mathsf{receive}_{P_i}, \mathsf{next}_{P_i}, \mathsf{output}_{P_i})$. A transcript τ records all messages exchanged during a protocol run. A run of the protocol with input I means the parties' respective inputs and public parameters are set to values represented by I, which satisfies implicit validity requirements.

Protocols must adhere to functionality requirements $\mathcal{F}$, constraining party outputs for specific inputs I, and security requirements $\mathcal{S}$, constraining message distributions conditioned on particular inputs. For each security requirement $\mathcal{S}$, a corresponding party is often assigned as being "concerned" with ensuring it. A protocol is deemed secure for a party P if all security requirements relevant to P are satisfied.

Definition 6 (Protocols and Parties). Given a protocol $\mathcal{P} = (\mathsf{setup}, (P_i)_{i=1}^n)$, functionality $\mathcal{F}$, input I, party P, index j, and index set $J \subseteq \{1, \ldots, n\}$:

- $\tau \leftarrow \mathcal{P}(I)$: Transcript obtained by running $\mathcal{P}$ with input I.
- $\mathcal{P}_{P_j \Longrightarrow P}$: The protocol after replacing party P_j with P in $\mathcal{P}$.
- $\mathcal{P}_{J \Longrightarrow P}$: The protocol obtained by merging all parties $\{P_j\}_{j \in J}$ into a single implementation of P.
- If any party sends the special symbol $\perp$ as a message, the protocol terminates immediately, indicating a violation of functionality.
- P maintains $\mathcal{F}$ for P_j in $\mathcal{P}$ if $\mathcal{P}_{P_j \Longrightarrow P}$ satisfies $\mathcal{F}$ with negligible probability of failure over the parties' and setup procedure's random coins for any fixed input.

When $\mathcal{F}$, P'_j, and $\mathcal{P}$ are understood from context, we simply state that P maintains functionality.

[1]The message schedule determines how many messages a party must receive from each other party before sending its message.

Next, we recall the definition of a cryptographic reverse firewall.

Definition 7 (Cryptographic reverse firewall [MSD15]). A cryptographic reverse firewall (RF) is a stateful algorithm $\mathcal{W}$ that takes its state and a message as input and outputs an updated state and message. For simplicity, we do not write the state of $\mathcal{W}$ explicitly. For a party $P = (\mathsf{receive}, \mathsf{next}, \mathsf{output})$ and reverse firewall $\mathcal{W}$, the composed party with RF is defined as,

$$\begin{aligned}\mathcal{W} \circ P :=& (\mathsf{receive}_{\mathcal{W}\circ P}(\sigma, m) = \mathsf{receive}_P(\sigma, \mathcal{W}(m)),\\ &\mathsf{next}_{\mathcal{W}\circ P}(\sigma) = \mathcal{W}(\mathsf{next}_P(\sigma)),\\ &\mathsf{output}_{\mathcal{W}\circ P}(\sigma) = \mathsf{output}_P(\sigma))\end{aligned}$$

When the composed party engages in a protocol, the state of $\mathcal{W}$ is initialized to the public parameters σ. If $\mathcal{W}$ is meant to be composed with a party P, we call it reverse firewall for P.

With the relevant concepts established, we now define the properties that a trojan-resilient reverse firewall must fulfill:

Definition 8 (Robust Functionality-maintaining RFs). Let $\mathcal{W}$ be a reverse firewall with circuit specification $\Gamma_{\mathcal{W}}$, and let $\mathcal{C}_{\mathcal{W}}$ be the circuit obtained from the circuit transformation algorithm of the trojan protection scheme, i.e., $\mathcal{C}_{\mathcal{W}} = (\mathcal{M} \Longleftrightarrow D_1, \cdots, D_\ell)(\cdot)$. For any party P, let $\mathcal{C}^1_{\mathcal{W}} \circ P = \mathcal{C}_{\mathcal{W}} \circ P$, and for $k \geq 2$, let $\mathcal{C}^k_{\mathcal{W}} \circ P = \mathcal{C}_{\mathcal{W}} \circ (\mathcal{C}^{k-1}_{\mathcal{W}} \circ P)$. For a protocol $\mathcal{P}$ that satisfies some functionality requirements $\mathcal{F}$, we say that a reverse firewall $\mathcal{W}$ maintains $\mathcal{F}$ for any party P in $\mathcal{P}$ if $\mathcal{C}^k_{\mathcal{W}} \circ P$ maintains $\mathcal{F}$ for P in $\mathcal{P}$ for any polynomially bounded $k \geq 1$. When $\mathcal{F}$, P, and $\mathcal{P}$ are clear from context, we simply say that the firewall $\mathcal{W}$ maintains functionality.

Naturally, a firewall is only valuable if it offers a tangible benefit. The primary motivation for deploying a reverse firewall is to maintain a protocol's security, even in the event of a party's compromise (i.e, when the party's implementation is tampered with). In this work we define a stronger notion of *robust security preservation*, which extends this guarantee by requiring the protocol to remain secure not only when a party is compromised, but also when the reverse firewall's implementation is outsourced to a potentially malicious manufacturer.

Definition 9 (Robust Security-preserving RFs). For a protocol $\mathcal{P} = (\mathsf{setup}, (\mathsf{next}_{\mathsf{P}_\mathsf{i}}, \mathsf{receive}_{\mathsf{P}_\mathsf{i}}, \mathsf{output}_{\mathsf{P}_\mathsf{i}})_{\mathsf{i}=1}^{\mathsf{n}})$ that satisfies some security requirements $\mathcal{S}$ and functionality $\mathcal{F}$ and a reverse firewall $\mathcal{W}^{\mathtt{Tr-RF}}$ with circuit specification $\Gamma_{\mathcal{W}}$,

1. $\mathcal{W}^{\mathtt{Tr-RF}}$ is *robust strongly security preserving* for $\mathcal{S}$ for party P_j in $\mathcal{P}$ if the protocol $\mathcal{P}_{P_j \Longrightarrow \mathcal{C}_{\mathcal{W}} \circ P^*_{\mathcal{A}}}$ satisfies $\mathcal{S}$, for any probabilistic polynomial-time $P^*_{\mathcal{A}}$ (potentially malicious implementation of P_j). Here, $\mathcal{C}_{\mathcal{W}}(\cdot) = (\mathcal{M} \Longleftrightarrow D_1, \cdots, D_\ell)(\cdot)$ as described above.

2. $\mathcal{W}^{\mathtt{Tr-RF}}$ is *robust security preserving* for party P_j in $\mathcal{P}$ against $\mathcal{F}$-maintaining adversaries if the protocol $\mathcal{P}_{P_j \Longrightarrow \mathcal{C}_{\mathcal{W}} \circ P^*_{\mathcal{A}}}$ satisfies $\mathcal{S}$ for any probabilistic polynomial-time $P^*_{\mathcal{A}}$ that maintains functionality $\mathcal{F}$.

When the implementations of parties can be arbitrarily tampered with, the corresponding security notion is referred to as *strong* (e.g., robust strong security-preserving or robust strong exfiltration-resistant). Conversely, if the tampering preserves the functionality of the parties, the term "strong" is omitted.

When $\mathcal{S}, P_j, \mathcal{P}$ and $\mathcal{F}$ are clear from context, we simply say that $\mathcal{W}^{\mathtt{Tr-RF}}$ is robust strongly security-preserving and robust security-preserving respectively.

Lastly, we require a trojan-resilient RF to meet the *robust exfiltration-resistance* property as defined below.

Definition 10 (Robust Exfiltration-resistant RFs). For a protocol $\mathcal{P}$ satisfying functionality $\mathcal{F}$ and a reverse firewall $\mathcal{W}^{\mathtt{Tr-RF}}$ with circuit specification $\Gamma_{\mathcal{W}}$,

– $\mathcal{W}^{\mathtt{Tr-RF}}$ is *robust strongly exfiltration-resistant* for party P_i against party P_j in protocol $\mathcal{P}$, if no PPT adversary $\mathcal{A}$ achieves advantage that is non-negligible in the security parameter k in the game $\mathsf{ROBER}(\mathcal{P}, P_i, P_j, \Gamma_{\mathcal{W}}, \vec{m}, k)$, as shown in Figure 4. If P_j is empty, then we say that the firewall is robust strongly exfiltration resistant with respect to eavesdroppers.

– $\mathcal{W}^{\mathtt{Tr-RF}}$ is *robust exfiltration-resistant* for party P_i against party P_j in protocol $\mathcal{P}$ satisfying functionality $\mathcal{F}$, if no PPT adversary $\mathcal{A}$ achieves advantage that is non-negligible in the security parameter k in the game $\mathsf{ROBER}(\mathcal{P}, P_i, P_j, \Gamma_{\mathcal{W}}, \vec{m}, k)$, as shown in Figure 4. If P_j is empty, then we simply say that the firewall is robust exfiltration resistant with respect to eavesdroppers.

ROBER($\mathcal{P}$, P_i, P_j, $\Gamma_{\mathcal{W}}$, $\vec{m}$, k)

1. $\left((\mathcal{M}, \{\Gamma_{\mathcal{W},i}\}_i), \vec{m}'\right) \leftarrow (\mathsf{TR}_1(1^k, \ell, \Gamma_{\mathcal{W}}), \mathsf{TR}_2(1^k, \ell, \vec{m}))$
2. $(D_1, \cdots, D_\ell, \bar{P}_i, \bar{P}_j, I) \leftarrow \mathcal{A}(1^k, \mathcal{M}, \{\Gamma_{\mathcal{W},i}\}_i)$
3. Set the initial state of the devices $\mathsf{Init}(\{D_i\}_i, \vec{m}')$
4. $b \xleftarrow{\$} \{0,1\}$;
5. If $b = 1$, set $P_i^* \leftarrow \mathcal{C}_{\mathcal{W}} \circ \bar{P}_i$
   where $\mathcal{C}_{\mathcal{W}}(\cdot) = (\mathcal{M} \Longleftrightarrow D_1, \cdots, D_\ell)[\vec{m}'](\cdot)$
6. Else, $P_i^* \leftarrow \Gamma_{\mathcal{W}} \circ P_i$.
7. $\tau^* \leftarrow \mathcal{P}_{P_i \Longrightarrow P_i^*, P_j \Longrightarrow \bar{P}_j}(I)$.
8. $b^* \leftarrow \mathcal{A}(\tau^*, \sigma_{\bar{P}_j})$.
9. If $b = b^*$ return 1; else return 0.

Figure 4: The Robust Exfiltration Resistance security game for a reverse firewall $\mathcal{W}^{\mathtt{Tr-RF}}$ with specification $\Gamma_{\mathcal{W}}$ for party P_i in protocol $\mathcal{P}$ against party P_j with input I.

In the first step of Game ROBER, the circuit transformation algorithm TR compiles the circuit specification $\Gamma_{\mathcal{W}}$ of the firewall $\mathcal{W}^{\mathtt{Tr-RF}}$ to produce $(\mathcal{M}, \{\Gamma_{\mathcal{W},i}\}_i, \vec{m}')$. This output is then provided to the malicious manufacturer $\mathcal{A}$, who generates a set of devices $\{D_i\}_i$ along with the tampered implementations $\bar{P}_i$ and $\bar{P}_j$ for parties P_i and P_j, respectively. The challenger then flips an unbiased coin b. If $b = 1$, the challenger replaces party P_i in the protocol $\mathcal{P}$ with the composed party $P_i^* = \mathcal{C}_{\mathcal{W}} \circ \bar{P}_i$, where $\bar{P}_i$ is the tampered implementation of P_i. If $b = 0$, the challenger replaces P_i with $P_i^* = \Gamma_{\mathcal{W}} \circ P_i$, the honest implementation of P_i. The protocol $\mathcal{P}$ is then executed with P_i replaced by P_i^* and P_j replaced by $\bar{P}_j$. The adversary is given the transcript τ^* of the protocol execution along with the state $\sigma_{\bar{P}_j}$ of the tampered party $\bar{P}_j$. The adversary wins the game if it can guess the bit b with a probability significantly better than $\frac{1}{2}$.

Robust Exfiltration-resistance implies Standard Exfiltration-Resistance. Our definition of robust exfiltration-resistance (ER) extends and generalizes the concept of standard exfiltration-resistance introduced in prior works. In standard ER, there are no circuit transformation steps; specifically, Step 1 in Figure 4 is omitted, and in Step 2, the adversary outputs only the tampered implementations $\bar{P}_i$ and $\bar{P}_j$ and the input I. Furthermore, in Step 5, the circuit $\mathcal{C}_{\mathcal{W}}$ is defined as $\mathcal{C}_{\mathcal{W}}(\cdot) = \Gamma_{\mathcal{W}}(\cdot)$.

Having introduced our new definitions for trojan-resilient RF, we present two essential lemmas associated with them. First, we show that trojan *robustness* along with standard *exfiltration-resistance* for a RF implies robust exfiltration-resistance.

Lemma 1. Let Π = *(TR, T)* be a trojan protection scheme that is (t, r, ϵ)-trojan robust, as defined in Definition 4. Also, let the firewall $\mathcal{W}^{\mathtt{Tr-RF}}$ with circuit specification $\Gamma_{\mathcal{W}}$ be (strongly) exfiltration-resistant for party P_i against party P_j in protocol $\mathcal{P}$ satisfying functionality $\mathcal{F}$. Then, the circuit $\mathcal{C}_{\mathcal{W}}$ (see Figure 4) derived by applying $\Pi = (\mathtt{TR}, \mathtt{T})$ to $\mathcal{W}^{\mathtt{Tr-RF}}$ is robust (strongly) exfiltration-resistant for party P_i against party P_j in the protocol $\mathcal{P}$ satisfying functionality $\mathcal{F}$.

Proof. To proof the above lemma, we define a sequence of hybrid games. In the following, let S_i be the event that the adversary wins in Game i.

Game 0: This game is the same as the real game $\mathsf{ROBER}(\mathcal{P}, P_i, P_j, \Gamma_{\mathcal{W}}, \vec{m}, k)$ (see Figure 4). Hence we have,

$$\Pr[S_0] = \mathsf{negl}(k)$$

Game 1: In this game, we replace the sub-devices D_i by the corresponding specifications $\widetilde{\Gamma_{W,i}}$ as shown in Figure 5. By Assumption 1, Game 1 is identical to the real game Game 0, i.e.,

$$\Pr[S_1] = \Pr[S_0]$$

$\mathsf{ROBER}^1(\mathcal{P},\ P_i,\ P_j,\ \Gamma_{\mathcal{W}},\ \vec{m},\ k)$

1. $\left((\mathcal{M}, \{\Gamma_{\mathcal{W},i}\}_i), \vec{m}'\right) \leftarrow (\mathsf{TR}_1(1^k, \ell, \Gamma_{\mathcal{W}}), \mathsf{TR}_2(1^k, \ell, \vec{m}))$
2. $(D_1, \cdots, D_\ell, \bar{P}_i, \bar{P}_j, I) \leftarrow \mathcal{A}(1^k, \mathcal{M}, \{\Gamma_{\mathcal{W},i}\}_i)$;
3. Set the initial state of the devices $\mathsf{Init}(\{\widetilde{\Gamma}_i\}_i, \vec{m}')$;
4. $b \xleftarrow{\$} \{0,1\}$;
5. If $b = 1, P_i^* \leftarrow \mathcal{C}_{\mathcal{W}} \circ \bar{P}_i$;
   where $\mathcal{C}_{\mathcal{W}}(\cdot) = (\mathcal{M} \Longleftrightarrow \widetilde{\Gamma_{W,1}}, \cdots, \widetilde{\Gamma_{W,\ell}})[\vec{m}'](\cdot)$;
6. Else, $P_i^* \leftarrow \Gamma_{\mathcal{W}} \circ P_i$;
7. $\tau^* \leftarrow \mathcal{P}_{P_i} \Longrightarrow_{P_i^*, P_j} \Longrightarrow_{\bar{P}_j} (I)$;
8. $b^* \leftarrow \mathcal{A}(\tau^*, \sigma_{\bar{P}_j})$;
9. If $b = b^*$ return 1; else return 0.

Figure 5: The modified Robust Exfiltration Resistance security game for a reverse firewall $\mathcal{W}^{\mathtt{Tr-RF}}$ with specification $\Gamma_{\mathcal{W}}$ for party P_i in protocol $\mathcal{P}$ against party P_j with input I. Here, the sub-devices D_i are replaced with their corresponding specifications $\widetilde{\Gamma_{W,i}}$.

Game 2: In this game, we replace the circuit specifications $\widetilde{\Gamma_{W,i}}$ with their corresponding *honest* specification $\Gamma_{W,i}$. In other words, in Line 3 of the ROBER game the initial state of each device D_i is initialized to $\mathsf{Init}(\{\Gamma_{W,i}\}_i, \vec{m}')$. Also, the circuit $\mathcal{C}_{\mathcal{W}}$ in Line 5 is set as $\mathcal{C}_{\mathcal{W}}(\cdot) = (\mathcal{M} \Longleftrightarrow \Gamma_{W,1}, \cdots, \Gamma_{W,\ell})[\vec{m}'](\cdot)$.

Claim 1.

$$|\Pr[S_2] - \Pr[S_1]| \leq \ell \cdot \epsilon(k)$$

Proof. The proof follows from a hybrid argument. In particular, for $i = 1, \cdots, \ell$, let us define the following sequences of hybrid games:

Game $1.i$*:* This game is identical to Game $1.i-1$, except that the first i sub-devices $\{D_j\}_{j=1}^{i}$ are replaced by the corresponding honest specification $\Gamma_{W,i}$. The rest of the devices $\{D_{i+1}, \cdots D_\ell\}$ are set as before, i.e., the specifications $\{\widetilde{\Gamma_{W,i+1}}, \cdots, \widetilde{\Gamma_{W,\ell}}\}$. See Figure 6 for details. Note that, in Game $1.\ell$ is the same as Game 2.

Claim 2.

$$|\Pr[S_{1.i}] - \Pr[S_{1.i-1}]| \leq \epsilon(k)$$

Proof. The proofs follows from the (t, r, ϵ)-trojan robustness of the trojan protection scheme Π (see Definition 4). The only distinction between Game $1.i-1$ and Game $1.i$ lies in whether $\widetilde{\Gamma_{W,i}}$ or $\Gamma_{W,i}$ is used to construct the circuit $\mathcal{C}_{\mathcal{W}}$. By the definition of trojan robustness, except with probability $\epsilon(k)$, $\Gamma_{W,i}$ and $\widetilde{\Gamma_{W,i}}$ are

functionality equivalent (provided the testing phase T passes). Therefore, any distinguisher capable of differentiating between these two games can be directly leveraged to violate the trojan robustness of Π. □

ROBER$^{1,i}(\mathcal{P}, P_i, P_j, \Gamma_{\mathcal{W}}, \vec{m}, k)$

1. $\left((\mathcal{M}, \{\Gamma_{\mathcal{W},i}\}_i), \vec{m}'\right) \leftarrow (\mathsf{TR}_1(1^k, \ell, \Gamma_{\mathcal{W}}), \mathsf{TR}_2(1^k, \ell, \vec{m}))$;
2. $(D_1, \cdots, D_\ell, \bar{P}_i, \bar{P}_j, I) \leftarrow \mathcal{A}(1^k, \mathcal{M}, \{\Gamma_{\mathcal{W},i}\}_i)$;
3. Set the initial state of the first i devices: $\mathsf{Init}(\{\Gamma_{W,\alpha}\}_{\alpha=1}^{i}, \vec{m}')$;
4. Set the initial state of the remaining devices: $\mathsf{Init}(\{\widetilde{\Gamma_{W,\beta}}\}_{\beta=i+1}^{\ell}, \vec{m}')$;
5. $b \xleftarrow{\$} \{0,1\}$;
6. If $b = 1$, $\quad P_i^* \leftarrow \mathcal{C}_{\mathcal{W}} \circ \bar{P}_i$;

   where $\mathcal{C}_{\mathcal{W}}(\cdot) = (\mathcal{M} \Longleftrightarrow \Gamma_{W,1}, \cdots, \Gamma_{W,i}, \widetilde{\Gamma_{W,i+1}}, \cdots, \widetilde{\Gamma_{W,\ell}})[\vec{m}'](\cdot)$;
7. Else, $P_i^* \leftarrow \Gamma_{\mathcal{W}} \circ P_i$;
8. $\tau^* \leftarrow \mathcal{P}_{P_i \Longrightarrow P_i^*, P_j \Longrightarrow \bar{P}_j}(I)$;
9. $b^* \leftarrow \mathcal{A}(\tau^*, \sigma_{\bar{P}_j})$;
10. If $b = b^*$ return 1; else return 0.

Figure 6: The modified Robust Exfiltration Resistance security game corresponding to Game 1.i.

□

Game 3 : This game is similar to Game 2, except that, when the bit $b = 1$, we set P_i^* as $P_i^* \leftarrow \Gamma_{\mathcal{W}} \circ \bar{P}_i$. In other words, we set $\mathcal{C}_{\mathcal{W}} = \Gamma_{\mathcal{W}}$.

Claim 3.

$$\Pr[S_3] = \Pr[S_2]$$

Proof. This follows from the fact that, $\Gamma_{W,1}, \cdots \Gamma_{W,\ell}$ are obtained by applying the circuit transformation algorithm TR on the circuit specification $\Gamma_{\mathcal{W}}$. Hence the circuit $(\mathcal{M} \Longleftrightarrow \Gamma_{W,1}, \cdots, \Gamma_{W,\ell})[\vec{m}'](\cdot)$ is functionally equivalent to the circuit specification $\Gamma_{\mathcal{W}}[\vec{m}](\cdot)$. □

Finally, note that Game 3 is the *standard exfiltration-resistance* security game of the RF $\mathcal{W}^{\mathtt{Tr-RF}}$ for party P_i against party P_j in the protocol $\mathcal{P}$. Hence, we have:

$$\Pr[S_3] = \mathsf{negl}(k)$$

.

The proof of Lemma 1 thus follows. □ □

Next, we show that robust exfiltration-resistance *implies* robust security-preservation for a reverse firewall when the underlying protocol has *simulation-based* (computational) security guarantees.

Lemma 2. Let $\mathcal{F}$ be any functionality and let $\mathcal{P}$ be a protocol for realizing $\mathcal{F}$ with abort in presence of malicious adversaries. Let $\mathcal{W}^{\mathtt{Tr-RF}}$ be a RF that is robust (strongly) exfiltration-resistant for party P_i against party P_j in protocol $\mathcal{P}$ satisfying functionality $\mathcal{F}$, then the firewall $\mathcal{W}^{\mathtt{Tr-RF}}$ is robust security-preserving for protocol $\mathcal{P}$ according to Definition 9.

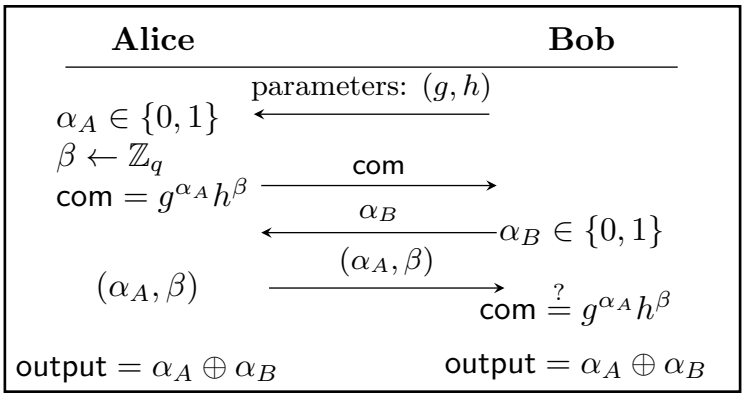

Figure 7: Blum's coin tossing protocol instantiated using the Pedersen commitment scheme.

Proof. The proof of this claim also follows from the trojan robustness property (see Def. 4) along with the result from [CGPS21]. In particular, [CGPS21] shows that standard (strong) exfiltration-resistance implies (strong) security preservation for a RF for simulation-based security notions. The trojan robustness guarantee implies that, with overwhelming probability in the robust exfiltration-resistance game, the circuit $\mathcal{C}_{\mathcal{W}}$ can be replaced with the original specification $\Gamma_{\mathcal{W}}$ of the firewall $\mathcal{W}^{\mathtt{Tr}-\mathtt{RF}}$. The claim then follows from the result of [CGPS21]. □

4 Trojan-resilient RFs for Coin Toss and Oblivious Transfer Protocol

In this section, we present trojan-resilient reverse firewalls for coin-tossing and oblivious transfer protocols, considering the security of these protocols in the *semi-honest* setting. For the coin-tossing protocol, we require that the parties' implementations be tampered with in a *functionality-maintaining* manner [MSD15], meaning that the tampered implementations do not disrupt the protocol's functionality. Most prior work on reverse firewalls adopts this tampering model, as it enables the attacker to exfiltrate information while remaining covert. In contrast, for the oblivious transfer protocol, we allow the adversary to tamper with the parties' implementations *arbitrarily*. We begin by presenting our two-party coin-tossing protocol, which is based on Blum's protocol [Blu81] and instantiated using Pedersen commitments. Following this, we describe the reverse firewalls designed for each party in the protocol. Finally, we propose a design model to upgrade these reverse firewalls into their trojan-resilient counterparts.

Two party Coin Tossing Protocol [Blu81]: We now present the Blum's semi-honest secure two-party coin-tossing protocol, illustrated in Figure 7. This protocol uses a bit commitment scheme $(\mathsf{Com}, \mathsf{Reveal})$ and enables both parties to agree on a single bit that is uniformly distributed at random. The protocol follows the classic *commit-and-open* paradigm: Alice first commits to her bit, after which Bob sends his bit. Alice then reveals her committed bit, and the final coin toss outcome is computed as the XOR of their two bits.

Reverse Firewall for the Coin-Tossing Protocol. Figure 8 illustrates the coin-tossing protocol integrated with reverse firewalls, denoted as $\mathcal{W}_A$ for Alice and $\mathcal{W}_B$ for Bob. This protocol further requires the bit commitment scheme to be *additively homomorphic*, a property already satisfied by Pedersen commitments.

- *Setup:* Alice and Bob agree on a security parameter k and two generators $(g, h) \in \mathbb{G}^2$, where $(\mathbb{G}, \cdot)$ is a multiplicative cyclic group in which the discrete logarithm problem is assumed to be hard.
- *Protocol Steps:*
  1. Alice selects a random bit $\alpha_A \in \{0,1\}$, random value $\beta \in \mathbb{Z}_q$, and sends the commitment $\mathsf{com} = g^{\alpha_A} h^{\beta}$.
  2. Alice's firewall $\mathcal{W}_A$ samples random values $r_1 \in \mathbb{Z}_q$ and $\alpha_1 \in \{0,1\}$. It re-randomizes the commitment as $\mathsf{com}' = \mathsf{com} \cdot g^{\alpha_1} h^{r_1}$ and forwards it to the outside network.

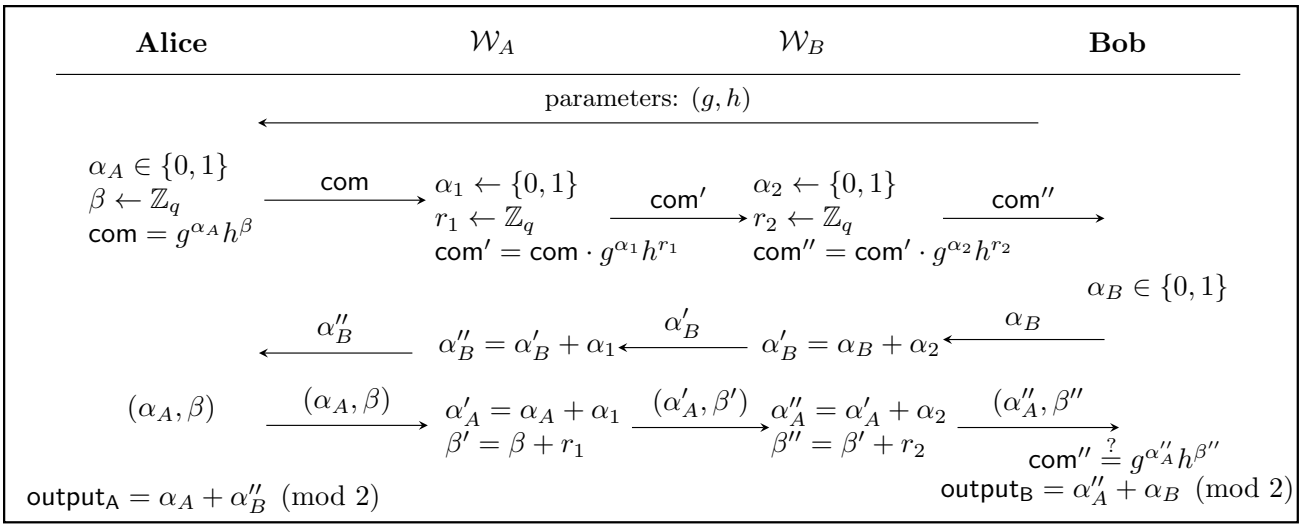

Figure 8: Reverse firewall enabled coin tossing protocol

3. Bob's firewall $\mathcal{W}_B$ further samples $r_2 \in \mathbb{Z}_q$ and $\alpha_2 \in \{0,1\}$. It re-randomizes the received commitment com' to $\mathsf{com}'' = \mathsf{com}' \cdot g^{\alpha_2} h^{r_2}$ and forwards it to Bob.
4. Bob then samples a random bit $\alpha_B \in \{0,1\}$ and sends it.
5. $\mathcal{W}_B$ re-randomizes Bob's bit using the earlier sampled value α_2 to obtain $\alpha'_B = \alpha_B + \alpha_2$ and forwards it to the outside network.
6. Upon receiving α'_B, $\mathcal{W}_A$ computes $\alpha''_B = \alpha'_B + \alpha_1$ and forwards it to Alice.
7. Upon receiving α''_B, Alice sends (α_A, β) as the decommitment. Further, it computes the output of the coin tossing protocol as $\mathsf{output_A} = (\alpha_A + \alpha''_B) \mod 2$.
8. $\mathcal{W}_A$ re-randomizes the tuple (α_A, β) to (α'_A, β') as: $\alpha'_A = \alpha_A + \alpha_1$, and $\beta' = \beta + r_1$ and forwards them to the outside network.
9. Upon receiving the tuple (α'_A, β'), $\mathcal{W}_B$ changes the tuple to (α''_A, β'') as follows: $\alpha''_A = \alpha'_A + \alpha_2$, and $\beta'' = \beta' + r_2$, and sends the pair (α''_A, β'') to Bob.
10. Bob verifies the commitment by checking if $\mathsf{com}'' \stackrel{?}{=} g^{\alpha''_A} h^{\beta''}$.
    If the verification is successful, he computes the coin toss result as $\mathsf{output_B} = (\alpha''_A + \alpha_B) \mod 2$. Otherwise, he aborts the protocol.

Theorem 2. Let $\prod$ be the reverse firewall enabled coin-tossing protocol depicted in Figure 8. The firewalls $\mathcal{W}_A$ and $\mathcal{W}_B$ are functionality maintaining. If the commitment scheme $\mathsf{COMMIT} = (\mathsf{Com}, \mathsf{Reveal})$ is perfectly hiding, computationally binding and is homomorphic with respect to the operations as shown in Figure 8, $\mathcal{W}_A$ and $\mathcal{W}_B$ preserve security and is exfiltration resistant.

Proof. First, we show that the firewalls $\mathcal{W}_A$ and $\mathcal{W}_B$ are *functionality maintaining* and hence preserves correctness of the protocol. The output of Alice is: $\mathsf{output_A} = (\alpha_A + \alpha''_B) \mod 2 = ((\alpha_A + \alpha_B) + (\alpha_1 + \alpha_2)) \mod 2$. Similarly, the output of Bob is: $\mathsf{output_B} = (\alpha''_A + \alpha_B) \mod 2 = ((\alpha_A + \alpha_B) + (\alpha_1 + \alpha_2)) \mod 2$. Hence, the output of both the parties are the same.

Exfiltration-resistance: First, we show that the firewall $\mathcal{W}_A$ of Alice is exfiltration-resistant *against* Bob in the above coin-tossing protocol $\prod$.

To prove exfiltration-resistance of $\mathcal{W}_A$, we first observe that the homomorphically evaluated commitment com' is uniformly distributed and also independent of the original commitment com sent by Alice. This is because the RF $\mathcal{W}_A$ chooses an independent bit $\alpha_1 \in \{0,1\}$ and randomness $r_1 \leftarrow \mathbb{Z}_q$ to homomorphically evaluate the original (potentially malicious) commitment string com. Hence, the firewall sanitizes the messages sent

across by Alice, even though the implementation of Alice may be corrupt. Since, the tampering of Alice is *functionality-maintaining*, its second message, i.e., the decommitment string is fixed, unless he can find an alternate opening for com, which by definition of binding is computationally hard. Hence, it follows that the reverse firewall for Alice is exfiltration-resistant against Bob in the protocol $\prod$.

Next, we show that the firewall $\mathcal{W}_B$ for Bob is exfiltration-resistant *against* Alice in the protocol $\prod$. First, note that the mauled commitment com'' is a uniformly random commitment to a uniformly random string. Since, the commitment scheme COMMIT is perfectly hiding, the bit α_B sampled by Bob is independent of the commitment com''. The firewall $\mathcal{W}_B$ also mauls the bit α_B using previously sampled bit α_2 (that was used to maul the commitment string com'), and hence the final bit that Alice outputs is random, irrespective of how α_B was chosen. Hence, $\mathcal{W}_B$ provides exfiltration-resistance for Bob against Alice.

Security Preservation: Now, we outline the simulation-based security proof for our proposed reverse firewall-equipped coin-tossing protocol, which builds upon Blum's coin-tossing protocol as described in [Lin17]. To prove the security preservation of the coin tossing protocol in the presence of a reverse firewall, we consider two different compositions of the reverse firewall. 1) Honest party composed with reverse firewall ($\mathcal{W} \circ P$), and 2) Tampered implementation of party composed with reverse firewall ($\mathcal{W} \circ \bar{P}$). We assume the tempering of the party has been done in a functionality-maintaining way. We build a simulator for an adversary which can observe the protocol transcripts after re-randomisation by the reverse firewall. In other words, this adversary can only see the transcript of composed party ($\mathcal{W} \circ P$ or $\mathcal{W} \circ \bar{P}$). Equivalently, this setup can be viewed as two parties running a coin-tossing protocol: Alice (with her reverse firewall $\mathcal{W}_A$) as P_1 and Bob (with his reverse firewall $\mathcal{W}_B$) as P_2.

Let $f_{\mathsf{rct}}(k, k) = (U_1, U_1)$ denote the ideal functionality of the coin-tossing protocol. Let $\mathcal{W}_A \circ P_1$ be the reverse firewall composed with Alice, and $\mathcal{W}_A \circ \bar{P}_1$ be the reverse firewall composed with functionality-maintaining tempered implementation of Alice. Now, we argue that due to the exfiltration-resistance property, the transcripts of the protocol due to $\mathcal{W}_A \circ P_1$ and $\mathcal{W}_A \circ \bar{P}_1$ are indistinguishable. To prove that the composed party preserves the security of the coin tossing protocol, we construct a simulator $\mathcal{S}$ that acts as an ideal-model adversary. The simulator externally interacts with a trusted party executing the ideal functionality and internally engages with a real-model adversary. Let $\mathcal{A}$ be a non-uniform probabilistic polynomial-time adversary.

Case1: P_2 is corrupted. The simulator $\mathcal{S}$ operates as follows:

1. $\mathcal{S}$ interacts with the ideal functionality f_{rct} and receives a bit b.
2. The simulator initialises a loop counter $i = 0$.
3. It randomly selects $\alpha_A \in_R \{0,1\}$ and $\beta \in_R \{0,1\}^k$ and computes $\mathsf{com} = \mathsf{Com}(\alpha_A; \beta)$. It then randomly selects $\alpha_1 \in_R \{0,1\}$ and $r_1 \in_R \{0,1\}^k$ and re-randomize the commitment as $\mathsf{com}' = \mathsf{com} \cdot g^{\alpha_1} \cdot h^{r_1}$. Now, it sends $\mathcal{A}$ the value com'.
4. If $\mathcal{A}$ responds with $\alpha_B = b + \alpha_A + \alpha_1 \pmod 2$, $\mathcal{S}$ forwards the pair $(\alpha_A + \alpha_1, \beta + r_1)$ to $\mathcal{A}$ and outputs whatever $\mathcal{A}$ outputs.
5. If $\mathcal{A}$ responds with $\alpha_B \neq b + \alpha_A + \alpha_1 \pmod 2$ and $i < k$, $\mathcal{S}$ increments i by 1 and returns to step (3).
6. If $i = k$, $\mathcal{S}$ outputs fail.

An iteration succeeds if and only if $\alpha_A + \alpha_1 + \alpha_B \pmod 2 = b$, where α_B is $\mathcal{A}$'s response to $\mathsf{Com}(\alpha_A + \alpha_1)$. We analyse the probability of this event:

$$
\begin{aligned}
&\mathsf{Pr}[\mathcal{A}(\mathsf{Com}(\alpha_A+\alpha_1))=\alpha_A+\alpha_1+b \pmod 2] \\
&\quad=\frac{1}{4}\cdot\mathsf{Pr}[\mathcal{A}(\mathsf{Com}(0))=b]+\frac{1}{4}\cdot\mathsf{Pr}[\mathcal{A}(\mathsf{Com}(1))=1\oplus b] \\
&\quad+\frac{1}{4}\cdot\mathsf{Pr}[\mathcal{A}(\mathsf{Com}(1))=1\oplus b]+\frac{1}{4}\cdot\mathsf{Pr}[\mathcal{A}(\mathsf{Com}(2))=b] \\
&\quad=\frac{1}{4}\cdot\mathsf{Pr}[\mathcal{A}(\mathsf{Com}(0))=b]+\frac{1}{4}\cdot(1-\mathsf{Pr}[\mathcal{A}(\mathsf{Com}(1)))=b] \\
&\quad+\frac{1}{4}\cdot(1-\mathsf{Pr}[\mathcal{A}(\mathsf{Com}(1)))=b]+\frac{1}{4}\cdot\mathsf{Pr}[\mathcal{A}(\mathsf{Com}(2))=b] \\
&\quad=\frac{1}{2}+\frac{1}{4}\cdot(\mathsf{Pr}[\mathcal{A}(\mathsf{Com}(0))=b]+\mathsf{Pr}[\mathcal{A}(\mathsf{Com}(2))=b] \\
&\qquad\qquad\qquad\qquad\qquad\qquad-2\cdot\mathsf{Pr}[\mathcal{A}(\mathsf{Com}(1))=b])
\end{aligned}
$$

Since the Pedersen commitment scheme is perfectly hiding and computationally binding, so the required probability of adversary outputting $\alpha_B=\alpha_A+\alpha_1+b \pmod 2$ is

$$
\mathsf{Pr}[\mathcal{A}(\mathsf{Com}(\alpha_A+\alpha_1))=\alpha_A+\alpha_1+b \pmod 2]=\frac{1}{2}
$$

Next, we demonstrate that conditioned on the event that $\mathcal{S}$ does not output fail, the distributions IDEAL and REAL are statistically close. In both the real and ideal executions, the bit α_B sent by $\mathcal{A}$ is entirely determined by $\alpha_A+\alpha_1$ and $\beta+r_1$. Specifically, we can express α_B as $\mathcal{A}(\mathsf{Com}(\alpha_A+\alpha_1;\beta+r_1))$. Consequently, both distributions take the form $(b,\mathcal{A}(\alpha_A+\alpha_1,\beta+r_1)),\alpha_A+\alpha_1,\beta+r_1$, where $b=\alpha_A+\alpha_1+\mathcal{A}(\mathsf{Com}(\alpha_A+\alpha_1;\beta+r_1)) \pmod 2$. The distinction between these distributions is as follows:

- **Real:** In a real execution, α_A, α_1 and β, r_1 are sampled uniformly.
- **Ideal:** In an ideal execution, b is randomly selected, and then α_A, α_1 and β, r_1 are sampled uniformly under the constraint that $\alpha_A+\alpha_1+\mathcal{A}(\mathsf{Com}(\alpha_A+\alpha_1;\beta+r_1)) \pmod 2=b$.

We evaluate the probability of every $(\alpha_A+\alpha_1,\beta+r_1)$ occurring under these distributions to establish their statistical closeness. Fixing $\hat{\alpha}_A$, $\hat{\alpha}_1$ and $\hat{\beta}$, $\hat{r}_1$, we note that in the real execution, $(\hat{\alpha}_A+\hat{\alpha}_1,\hat{\beta}+\hat{r}_1)$ occurs with probability exactly $2^{-(k+3)}$.

In the ideal execution, let $S_b=\{(\alpha_A+\alpha_1,\beta+r_1)\mid \alpha_A+\alpha_1+\mathcal{A}(\mathsf{Com}(\alpha_A+\alpha_1;\beta+r_1)) \pmod 2=b\}$. This set S_b contains all pairs $(\alpha_A+\alpha_1,\beta+r_1)$ that could produce an output b in the ideal execution (since $\mathcal{S}$ halts only when $\alpha_A+\alpha_1+\mathcal{A}(\mathsf{Com}(\alpha_A+\alpha_1;\beta+r_1)) \pmod 2=b$). We claim that the fixed pair $(\hat{\alpha}_A+\hat{\alpha}_1,\hat{\beta}+\hat{r}_1)$ appears in the ideal execution with probability

$$
\frac{1}{2}\cdot\frac{1}{|S_b|}.
$$

This holds because b is uniformly chosen by the trusted party (ideal functionality), and conditioned on not outputting fail, the simulator $\mathcal{S}$ samples uniformly from S_b. Next, we demonstrate that for every $b\in\{0,1\}$, the set S_b contains 2^{n+2} elements. By the hiding property of Pedersen commitments, we observe that for every b, the probability that $\mathsf{Pr}[\mathcal{A}(\mathsf{Com}(\alpha_A+\alpha_1))=\alpha_A+\alpha_1+b \pmod 2]$ is $\frac{1}{2}$. This probability matches the likelihood that $(\alpha_a+\alpha_1,\beta+r_1)\in S_b$, implying

$$
\frac{|S_b|}{2^{n+3}}=\frac{1}{2}
$$

and thus

$$|S_b| = 2^{n+2}$$

From this, we deduce that the pair $(\hat{\alpha}_A + \hat{\alpha}_1, \hat{\beta} + \hat{r}_1)$ appears with probability between $2^{-(n+3)}$. This probability is therefore equal to the probability of $(\hat{\alpha}_A + \hat{\alpha}_1, \hat{\beta} + \hat{r}_1)$ occurring in the real execution. Hence, the real and ideal distributions are statistically equal.

Case2: P_1 is corrupt. When P_1 is corrupted, the simulation should also consider the case that $\mathcal{A}$ does not reply with a valid message and so aborts. The simulator $\mathcal{S}$ can be constructed as follows:

1. $\mathcal{S}$ sends the value k externally to the trusted party responsible for computing f_{rct} and receives a bit b in return.
2. $\mathcal{S}$ initiates interaction with the adversary $\mathcal{A}$ and internally records the message com that $\mathcal{A}$ sends to P_1.
3. $\mathcal{S}$ simulates P_2 by first sending the bit $\alpha_B = 0$ to $\mathcal{A}$ and capturing its response. Then, $\mathcal{S}$ sends the bit $\alpha_B = 1$ to $\mathcal{A}$ and captures its corresponding response. Note that, in this case the value α_B has already been re-randomized by the reverse firewall of Bob. The following scenarios are considered:
   (a) If $\mathcal{A}$ responds with a valid decommitment $(\alpha_A + \alpha_1, \beta + r_1)$ such that $\mathsf{Com}(\alpha_A + \alpha_1; \beta + r_1) = \mathsf{com}$ in both cases, $\mathcal{S}$ sends the $\mathsf{continue}$ message to the trusted party. Additionally, $\mathcal{S}$ computes $\alpha_B = \alpha_A + \alpha_1 + b \pmod 2$, forwards this value to $\mathcal{A}$, and outputs the adversary's final response.
   (b) If $\mathcal{A}$ fails to provide a valid decommitment in either case, $\mathcal{S}$ sends the abort message to the trusted party. Subsequently, $\mathcal{S}$ generates a random bit α_B, passes it to $\mathcal{A}$, and outputs the adversary's final response.
   (c) If $\mathcal{A}$ produces a valid decommitment $(\alpha_A + \alpha_1, \beta + r_1)$ such that $\mathsf{Com}(\alpha_A + \alpha_1; \beta + r_1) = \mathsf{com}$ only when given α_B where $\alpha_A + \alpha_1 + \alpha_B \pmod 2 = b$, $\mathcal{S}$ sends $\mathsf{continue}$ to the trusted party. Then, $\mathcal{S}$ calculates $\alpha_B = \alpha_A + \alpha_1 + b \pmod 2$, provides this value to $\mathcal{A}$, and outputs its final response.
   (d) If $\mathcal{A}$ generates a valid decommitment $(\alpha_A + \alpha_1, \beta + r_1)$ such that $\mathsf{Com}(\alpha_a + \alpha_1; \beta + r_1) = \mathsf{com}$ only when given α_B where $\alpha_A + \alpha_1 + \alpha_B \pmod 2 \neq b$, $\mathcal{S}$ sends the abort message to the trusted party. Then, $\mathcal{S}$ computes $\alpha_B = \alpha_A + \alpha_1 + b + 1 \pmod 2$, delivers it to $\mathcal{A}$, and outputs the final response.

We demonstrate that the output distributions are identical by analyzing the following cases:

1. **Case $3a$: $\mathcal{A}$ always replies with a valid decommitment:** In this scenario, $\mathcal{A}$'s view during a real execution consists of a random bit α_B, and the honest P_2's output is $b = \alpha_A + \alpha_1 + \alpha_B \pmod 2$, where $\alpha_A + \alpha_1$ is the value committed in com. Since $\alpha_A + \alpha_1$ is fully determined by the commitment com prior to α_B being chosen by the composed $\mathcal{W} \circ P_1$, it follows that b is uniformly distributed.

   In contrast, in an ideal execution, the bit b is uniformly chosen. Here, $\mathcal{A}$'s view consists of $\alpha_B = \alpha_A + \alpha_1 + b \pmod 2$, and the honest P_2's output equals b. Since $\alpha_A + \alpha_1$ is completely determined by the commitment com before $\mathcal{A}$ receives any information about b, it follows that $\alpha_B = \alpha_A + \alpha_1 + b \pmod 2$ is uniformly distributed.

   In both scenarios, the bits b and α_B are uniformly distributed, constrained by $b + \alpha_B = \alpha_A + \alpha_1$. Thus, the joint distributions over $\mathcal{A}$'s output and the honest party's output are identical in the real and ideal executions.

2. **Case $3b$: $\mathcal{A}$ never replies with a valid decommitment.** In this scenario, $\mathcal{A}$'s view consists of a uniformly distributed bit, which is identical to its view in a real execution. Furthermore, the honest P_2 outputs $\perp$ in both the real and ideal executions with probability 1. Consequently, the joint distributions over $\mathcal{A}$'s output and the honest party's output are identical in the real and ideal executions.

3. **Case $3c$ and $3d$: $\mathcal{A}$ replies with a valid decommitment for exactly one value** $\hat{\alpha}_B \in \{0, 1\}$. Let $\alpha_A + \alpha_1$ denote the value committed in the commitment com sent by $\mathcal{A}$. Since $\mathcal{A}$ is deterministic and this is the first message, $\alpha_A + \alpha_1$ is fixed. In the real execution:

- If P_2 sends $\hat{\alpha}_B$, then $\mathcal{A}$ replies with a valid decommitment, and the honest P_2 outputs $b = \alpha_A + \alpha_1 + \hat{\alpha}_B \pmod 2$.
- If P_2 sends $\hat{\alpha}_B \oplus 1$, then $\mathcal{A}$ does not reply with a valid decommitment, and P_2 outputs $\perp$.

Now, consider the ideal execution:

- If $b + \alpha_A + \alpha_1 \pmod 2 = \hat{\alpha}_B$, then $\mathcal{S}$ provides $\mathcal{A}$ with the bit $\hat{\alpha}_B$. In this case, $\mathcal{A}$ replies with a valid decommitment, and the honest P_2 outputs $b = \alpha_A + \alpha_1 + \hat{\alpha}_B \pmod 2$.
- If $b + \alpha_A + \alpha_1 \pmod 2 = \hat{\alpha}_B \oplus 1$, then $\mathcal{S}$ provides $\mathcal{A}$ with the bit $\hat{\alpha}_B \oplus 1$. In this case, $\mathcal{A}$ does not reply with a valid decommitment, and P_2 outputs $\perp$.

Thus, the distribution of $\mathcal{A}$'s view and P_2's output is identical in both the real and ideal executions. This proves the security of Alice in the presence of Alice's reverse firewall ($\mathcal{W}_A$) against a corrupt Bob. Similarly, it also proves the security of Bob in the presence of Bob's reverse firewall ($\mathcal{W}_B$) against a corrupt Alice. This concludes the proof of the theorem 2.

□

4.1 Trojan-resilient Reverse Firewall for Coin Tossing Protocol

This section outlines techniques to transform a legacy reverse firewall (RF) into a trojan-resilient RF. Our approach adapts the methodology from [DFS16] to protect reverse firewalls from trojan insertion. A trojan-resilient reverse firewall consists of two components: Circuit Transformation (TR) and the Testing Algorithm (T). Circuit transformation uses secure multiparty computation (MPC) protocols, such as those in Algorithms 1, 2, 3, and 4. These algorithm internally uses different functions like, BEAVER.TRIPLE(.)- takes a modulus q as input and outputs a, b, c such that $c = a * b$, SECRET.SHARE(.)- takes a values x as input and outputs shares of x as $[x]_p$ such that RECONSTRUCT($[x]_p$) gives x, MULT(.)- performs secure multiplication by taking two shares and a modulus as input and outputs the share of the multiplied values, and MOD.INVERSE(.) - takes x and the modulus as input and outputs x^{-1} to securely perform operations. The original circuit, Γ, is transformed into three smaller circuits, $\Gamma_0, \Gamma_1, \Gamma_2$, which collaboratively simulate Γ through a passively secure 3-party protocol. Inputs are secret-shared by a trusted master node (M), and the mini-circuits are fabricated by an untrusted manufacturer. The security of the trojan-resilient RF is captured through a robustness game (Figure 3), modeling a malicious manufacturer inserting trojans.

We propose our design model for trojan-resilient reverse firewalls below: In our this model, the master circuit M can sample a random value and distribute it to all the mini-circuits ($\Gamma_0, \Gamma_1, \Gamma_2$). This setup assumes that the master circuit is capable to communicate large values to the mini-circuits, providing robust security with minimal computation overhead. Besides, we provide an alternate design model with very small computation overhead on master circuit, in Appendix A but comes at the cost of one extra trust assumption that one among the three mini-circuit is honest. Now, we build trojan-resilient $\mathcal{W}_A$ and $\mathcal{W}_B$ below:

4.1.1 Trojan-Resilient $\mathcal{W}_A$

In the coin-tossing protocol, the reverse firewall $\mathcal{W}_A$ ① computes randomized commitment com′, ② computes α''_B and ③ computes α'_A and β'. Below, we describe the functionality of master and mini-circuits for these computation.

1. Distributed computation of com′:

- **Master Circuit** (M): It randomly selects a 16-byte Advanced Encryption Standards (AES) key k_1. Then it chooses two 16-byte input strings, let's say str1 and str2. It then generates the pseudo-random value, $r_1 = AES(k_1, \mathsf{str1})$. After that, it generates another random bit, $\alpha_1 = AES(k_1, \mathsf{str2}) \bmod 2$ and Secret shares the commitment com as $[\mathsf{com}]_p$ and shares the values r_1 and α_1 as $[r_1]_p$, $[\alpha_1]_p$ respectively among the mini-circuits.

Algorithm 1 Secure addition Protocol

Require: share $[x]_p$, share $[r]_p$ and modulus q.
Ensure: share $[x+r]_p$.
1: **return** $([x]_p + [r]_p) \pmod q$

Algorithm 2 Secure Multiplication Protocol

Require: share $[x]_p$, share $[r]_p$ and modulus q.
Ensure: share $[xr]_p$.
1: $(a, b, c) \leftarrow$ Beavers.Triple(q)
2: $[a]_p \leftarrow$ Secret.Share(a)
3: $[b]_p \leftarrow$ Secret.Share(b)
4: $[c]_p \leftarrow$ Secret.Share(c)
5: $[\delta]_p = [x]_p - [a]_p$
6: $[\epsilon]_p = [r]_p - [a]_p$
7: $\delta =$ Reconstruct($[\delta]_p$)
8: $\epsilon =$ Reconstruct($[\epsilon]_p$)
9: $[z]_p = \epsilon \cdot [a]_p + \delta \cdot [b]_p + [c]_p$
10: $xr =$ Reconstruct($[z]_p$) $+ \delta * \epsilon$
11: $[xr]_p \leftarrow$ Secret.Share(xr)
12: **return** $[xr]_p$

- **Mini Circuit** (Γ^i): Each mini-circuit invokes the public exponentiation protocol (Algorithm 3) and computes: $[g^{\alpha}]_p$ and $[h^{r_1}]_p$. It the invokes the secure multiplication protocol (Algorithm 2) and computes: $[g^{\alpha_1} \cdot h^{r_1}]_p$ and uses the result from the previous step and the shared value $[\mathsf{com}]_p$ to compute: $[\mathsf{com} \cdot g^{\alpha_1} \cdot h^{r_1}]_p$.
- **Master Circuit** (M): It now performs reconstruction over $[\mathsf{com} \cdot g^{\alpha_1} \cdot h^{r_1}]_p$ to produce the final value $\mathsf{com} \cdot g^{\alpha_1} \cdot h^{r_1}$.

2. Distributed computation of α''_B:

- **Master Circuit** (M): It generates shares of α'_B and α_1 as $[\alpha'_B]_p$ and $[\alpha_1]_p$ and sends it to the mini-circuits.
- **Mini Circuits**(Γ^i): Each mini circuit invokes secure addition protocol (Algorithm 1) and returns share $[\alpha'_B + \alpha_1]_p$ to master circuit.
- **Master Circuit**($\mathcal{M}$): Finally, it runs a reconstruction over the values received from the mini-circuits to compute α''_B.

3. Distributed Computation of α'_A and β':

- **Master Circuit** (M): It generates shares of α_A and α_1 as $[\alpha'_B]_p$ and $[\alpha_1]_p$ and shares of β and r_1 as $[\beta]_p$ and $[r_1]_p$ sends them to the mini-circuits.
- **Mini Circuits**(Γ^i): Each mini circuit invokes secure addition protocol (Algorithm 1) over input pair $([\alpha'_B]_p, [\alpha_1]_p)$ and $([\beta]_p, [r_1]_p)$ and returns share $[\alpha_A + \alpha_1]_p$ and $[\beta + r_1]_p$ to master circuit.
- **Master Circuit**($\mathcal{M}$): Finally, it runs the reconstruction over share $[\alpha_A + \alpha_1]_p$ and $[\beta + r_1]_p$ independently to compute α'_A and β'.

4.1.2 Trojan-resilient $\mathcal{W}_B$

The reverse firewall $\mathcal{W}_B$ implements the computation of com'', α'_B, α''_A and β''. However, these computation follows similar steps as the computations by $\mathcal{W}_A$, so we do not discuss $\mathcal{W}_B$ in detail here.

Theorem 3. Let Σ be the coin tossing protocol which is exfiltration-resistant and $\Pi = (\mathsf{TR}, \mathsf{T})$ be a trojan protection scheme that is (t, r, ϵ)-trojan robust, as defined in Definition 4. For reverse firewalls $\mathcal{W}_A$ and $\mathcal{W}_B$ if transformed into $\mathcal{W}_A^{\mathtt{Tr-RF}}$ and $\mathcal{W}_B^{\mathtt{Tr-RF}}$ by applying Π on $\mathcal{W}_A$ and $\mathcal{W}_B$ as shown above, then $\mathcal{W}_A^{\mathtt{Tr-RF}}$ and $\mathcal{W}_B^{\mathtt{Tr-RF}}$ are robust-exfiltration resistant.

Algorithm 3 Public Exponentiation Protocol

Require: base share $[x]_p$ of party p, public exponent y and modulus q.
Ensure: share $[x^y]_p$ of party p.
1: Generate a random value $r \in \{1, 2, \ldots, q-1\}$
2: $[r]_p \leftarrow$ Secret.Share(r)
3: $[r^y]_p \leftarrow$ Secret.Share(r^y)
4: $[xr]_p =$ Multiply$([x]_p, [r]_p, q)$
5: $xr =$ Reconstruct$([xr]_p)$
6: $[r^{-y}]_p =$ Secure.Invert$([r^y]_p, q)$
7: $[x^y]_p = (xr)^y \cdot [r^{-y}]_p$
8: **return** $[x^y]_p$

Algorithm 4 Private Inverse Protocol

Require: secret share $[x]_p$, and modulus q.
Ensure: share $[x^{-1}]_p$.
1: Generate a random value $r \in \{1, 2, \ldots, q-1\}$.
2: $r^{-1} =$ Mod.Inverse(r, q)
3: **while** $r^{-1} ==$ None **do**
4: Generate a random value $r \in \{1, 2, \ldots, q-1\}$.
5: $r^{-1} =$ Mod.Inverse(r, q)
6: $[r]_p \leftarrow$ Secret.Share(r)
7: $[xr]_p =$ Mult$([x]_p, [r]_p, q)$
8: $xr =$ Reconstruct$([xr]_p)$
9: $(xr)^{-1} =$ Mod.Inverse(xr, q)
10: **if** $(xr)^{-1} ==$ None **then**
11: goto step 1.
12: $[x^{-1}]_p = (xr)^{-1} \cdot [r]_p$
13: **return** $[x^{-1}]_p$

Proof. The proof for above theorem follows from Theorem 2 and Lemma 1. □

Theorem 4. Let Σ be the coin tossing protocol and $\mathcal{W}_A$ and $\mathcal{W}_B$ be the robust exfiltration-resistant reverse firewalls for Alice and Bob respectively, then $\mathcal{W}_A$ and $\mathcal{W}_B$ are also robust-security-preserving RFs.

Proof. The proof for above theorem follows from Theorem 3 and Lemma 2. □

4.2 Trojan-resilient Reverse Firewall for Oblivious Transfer Protocol

This section provides a brief description of the reverse firewall for oblivious transfer protocol as given in [MSD15], followed by its trojan-resilient instantiation.

OT protocol [MSD15]. Alice has inputs (m_0, m_1), and Bob has a choice bit b as input. Bob randomly samples generators $(g, c) \xleftarrow{\$} (\mathbb{G} \setminus \{1_\mathbb{G}\})^2$ and $y \xleftarrow{\$} \mathbb{Z}_q$. It computes and sends $(g, c, d = g^y, h = c^y g^b)$ to Alice. Then, Alice randomly samples $(r_0, s_0, r_1, s_1) \xleftarrow{\$} \mathbb{Z}_q^4$ and computes $(u_i, e_i)_{i=0}^1 \leftarrow (g^{r_i} c^{s_i}, d^{r_i}(h/g^i)^{s_i} m_i)_{i=0}^1$. It forwards $(u_i, e_i)_{i=0}^1$ to Bob. At the end, Bob computes $m_b = e_b / u_b^y$ to retrieve message as per his choice bit.

Bob's Firewall: To re-randomize (g, c, d, h), the firewall samples $\alpha \xleftarrow{\$} \mathbb{Z}_q^*$ and $(x', y') \xleftarrow{\$} \mathbb{Z}_q^2$. It computes $g' \leftarrow g^\alpha$, $c' \leftarrow c^\alpha g'^{x'}$, $d' \leftarrow d^\alpha g'^{y'}$, and $h' \leftarrow h^\alpha c^{\alpha y'} d^{\alpha x'} g'^{x'y'}$. It forwards these values to Alice. Upon receiving (u_0, e_0, u_1, e_1), the firewall de-randomizes by computing $e_0' \leftarrow e_0 / u_0^{y'}$, and $e_1' \leftarrow e_1 / u_1^{y'}$. It then forwards (u_0, e_0', u_1, e_1') to Bob.

Alice's Firewall: Alice's firewall checks if $g = 1_\mathbb{G}$ and aborts if true. Otherwise, forwards (g, c, d, h) to Alice. When Alice revert back with its computation, it re-randomizes (u_0, e_0, u_1, e_1) by sampling $(r_0', s_0', r_1', s_1') \xleftarrow{\$} \mathbb{Z}_q^4$ and computing:

$$(u_i', e_i')_{i=0}^1 \leftarrow (u_i g^{r_i'} c^{s_i'}, e_i d^{r_i'} (h/g^i)^{s_i'})_{i=0}^1.$$

The firewall forwards $(u'_i, e'_i)_{i=0}^{1}$ to Bob. We revisit following theorems from [MSD15] to claim the correctness and security guarantees.

Theorem 5. Bob's reverse firewall as described above maintains correctness, is strongly security-preserving and strongly exfiltration-resistant if chosen-base DDH with a hint game is hard in $\mathbb{G}$.

Theorem 6. Alice's reverse firewall, as described above, maintains correctness and is strongly exfiltration-resistant against an eavesdropper if DDH is hard in $\mathbb{G}$. The composed firewall (Bob's firewall and Alice's firewall) also preserves security against Bob.

Trojan-resilient RF for OT protocol: We implement Bob's reverse firewall with a master circuit ($\mathcal{M}$) and three mini-circuits ($\Gamma^0, \Gamma^1, \Gamma^2$), operating under the trojan-resilient model. We provide the step-wise computations of Alice's RF and Bob's RF below.

4.2.1 Trojan-Resilient $\mathcal{W}_B$ for OT

In the oblivious transfer protocol, the reverse firewall $\mathcal{W}_B$ ① computes g', ② computes c', ③ computes d' and, ④ computes h'. Below, we describe the functionality of master and mini-circuits for these computations.

1. Distributed computation of g', c', d' and h':

   - **Master Circuit** (M): It randomly selects a $\alpha \in \mathbb{Z}_p^*$ and $(x', y') \xleftarrow{\$} \mathbb{Z}_p^2$. It secret shares g, c, d and h as $[g]_p$, $[c]_p$, $[d]_p$ and $[h]_p$ respectively among mini-circuits. It then shares α, x', y' to all mini-circuits.
   - **Mini Circuit** (Γ^i): Each mini-circuit invokes the public exponentiation protocol (Algorithm 3) as $\textsc{Exp}([g]_p, \alpha)$, $\textsc{Exp}([c]_p, \alpha)$, $\textsc{Exp}([d]_p, \alpha)$, and $\textsc{Exp}([h]_p, \alpha)$ to compute $[g^\alpha]_p$, $[c^\alpha]_p$, $[d^\alpha]_p$ and $[h^\alpha]_p$ respectively. Now, they invoke public exponentiation protocol as $\textsc{Exp}([g^\alpha]_p, x')$, $\textsc{Exp}([g^\alpha]_p, y')$, $\textsc{Exp}([c^\alpha]_p, y')$, $\textsc{Exp}([d^\alpha]_p, x')$ and $\textsc{Exp}([g^\alpha]_p, x'y')$ to computes $[g^{\alpha x'}]_p$, $[g^{\alpha y'}]_p$, $[c^{\alpha y'}]_p$, $[d^{\alpha x'}]_p$ and $[g^{\alpha x' y'}]_p$. Afterwards, each mini-circuit invokes secure multiplication protocol as $\textsc{Mult}([c^\alpha]_p, [g^{\alpha x'}]_p)$, $\textsc{Mult}([d^\alpha]_p, [g^{\alpha y'}]_p)$, $\textsc{Mult}([h^\alpha]_p, [c^{\alpha y'}]_p)$, and $\textsc{Mult}([d^{\alpha x'}]_p, [g^{\alpha x' y'}]_p)$ to compute shares of c', d', h'. Note that the shares of h' is computed by appying another $\textsc{Mult}(.)$ on the results of last two multiplications.
   - **Master Circuit** (M): It now performs reconstruction over shares like, $g' = \textsc{reconstruct}([g^\alpha]_p)$. to produce the final value g', c', d' and h'.

2. Distributed computation of e'_0 and e'_1:

   - **Master Circuit** (M): It generates shares of e_0, e_1 and u_0, u_1 as $[e_0]_p$, $[e_1]_p$ and $[u_0]_p$, $[u_1]_p$ respectively and sends it to the mini-circuits along with the value y'.
   - **Mini Circuits**(Γ^i): Each mini circuit invokes secure public exponentiation protocol (Algorithm 3) as $\textsc{Exp}([u_0]_p, y')$ and $\textsc{Exp}([u_1]_p, y')$ to compute $[u_0^{y'}]_p$ and $[u_1^{y'}]_p$. After that, they perform private inverse protocol (Algorithm 4) as $\textsc{Inv}([u_0^{y'}]_p)$ and $\textsc{Inv}([u_1^{y'}]_p)$ to compute $[u_0^{-y'}]_p$ and $[u_1^{-y'}]_p$. Finally, each mini-circuit perform secure multiplication as $\textsc{Mult}([e_0]_p, [u_0^{-y'}]_p)$ and $\textsc{Mult}([e_1]_p, [u_1^{-y'}]_p)$ to compute $[e'_0]_p$ and $[e'_1]_p$.
   - **Master Circuit**($\mathcal{M}$): Finally, it runs a reconstruction over the values received from the mini-circuits to compute e'_0 and e'_1.

4.2.2 Trojan-Resilient $\mathcal{W}_A$ for OT

The trojan-resilient Alice's firewall is implemented using a master circuit and three mini-circuits. While there are replicas of mini-circuits performing identical computations and participating in majority voting, we focus on a single replica to illustrate the computation steps.

The master circuit first checks if $g = 1_{\mathbb{G}}$ and aborts if true; otherwise, it forwards (g, c, d, h) to Alice. Upon receiving (u_0, e_0, u_1, e_1) from Alice, the master circuit randomly selects (r'_0, s'_0, r'_1, s'_1). To compute u'_i, it secret-shares g and c, along with exponents r'_i and s'_i, respectively, with all mini-circuits. Using these shares, the mini-circuits perform secure multiplication to compute shares of $g^{r'_i}c^{s'_i}$. The secret shares of u_i are then combined with these results to compute shares of $u_i g^{r'_i} c^{s'_i}$. For the second part, the master circuit first computes shares of g raised to the power i using the public exponentiation algorithm. It then computes the inverse of g^i using the private inverse protocol and performs secure multiplication with the shares of h to compute shares of h/g^i. Subsequently, it uses public exponentiation on the shares of h/g^i with the exponent s'_i to compute shares of $(h/g^i)^{s'_i}$. Parallelly, it computes shares of $d^{r'_i}$ and performs secure multiplication to obtain shares of $d^{r'_i}(h/g^i)^{s'_i}$. Finally, the master circuit generates shares of e_i and combines them with the previously computed shares of $d^{r'_i}(h/g^i)^{s'_i}$ using secure multiplication to compute shares of $e_i d^{r'_i}(h/g^i)^{s'_i}$.

Theorem 7. Let Σ be the oblivious transfer protocol which is exfiltration-resistant (as per Theorem 6 and 5) and the reverse firewall $\mathcal{W}$ for both Alice and Bob are transformed into $\mathcal{W}^{\mathtt{Tr-RF}}$ applying Π on $\mathcal{W}$ as shown above, then $\mathcal{W}^{\mathtt{Tr-RF}}$ is robust-exfiltration resistant.

Proof. The proof for above theorem follows from Lemma 1. □

Theorem 8. Let Σ be the oblivious transfer protocol and $\mathcal{W}_A$ and $\mathcal{W}_B$ be the robust exfiltration-resistant reverse firewalls for Alice and Bob respectively, then $\mathcal{W}_A$ and $\mathcal{W}_B$ are also robust-security-preserving RFs.

Proof. The proof for above theorem follows from Lemma 2. □

5 Implementation

We implement a Trojan-resilient reverse firewall for coin tossing and oblivious transfer protocols on Open-Portable Trusted Execution Environment (OP-TEE), writing approx 5000 lines of code in C language, ensuring the communication among mini circuits and master circuit through an interconnected Local Area Network.

Practical Realization. The Trojan protection scheme in Private Circuit III employs different types of circuits, each with specific functionality. We expand more on *master circuit*, *sub-circuits*, and *mini-circuits* and provide the realizations of these circuits:

- *Master Circuit:* The master component, denoted as M, acts as the sole trusted hardware element in the architecture, essential for achieving Trojan-resilience. To maintain its integrity, M must either be manufactured by a trustworthy provider or be easily verifiable using reactive Trojan detection techniques, necessitating a minimal gate count in its design. The master circuit is responsible for directly interfacing with the adversary, managing the processes of secret sharing and reconstruction, and conducting majority voting on the outputs from the sub-circuits. In our implementation, M is simulated using the Raspberry Pi 4 (RPI4) Model B. The RPI4 is configured with the Rasbian operating system, which includes the `mbedtls` library for cryptographic operations and standard Linux libraries such as `arpa` to facilitate communication between devices via socket programming.
- *Sub-Circuit:* The *sub-circuits*, represented as Γ_i, consist of λ independent instances that execute a semi-honest three-party computation protocol on a given input. This setup ensures that the shares used in the three-party computation within each Γ_i are isolated and independent from those in other sub-circuits. Our methodology employs an efficient semi-honest multi-party computation protocol, incorporating operations such as secure addition, secure multiplication, secure public exponentiation, and secure private exponentiation. The semi-honest model provides sufficient passive security for our purposes, as the inclusion of a testing phase ensures the protocol executes correctly with high probability. For implementation, the sub-circuits are deployed on Raspberry Pi 3 (RPI3) devices, utilizing the Open Portable Trusted Execution Environment (OP-TEE) framework to enhance isolation and security.

Protocol	(1) Time (ms)	(2) Time (ms)
Addition Protocol	1.3	27.95
Multiplication Protocol	3.7	63.45
Exponentiation Protocol	7.5	69.89

Table 1: Execution time of (1) local operations, (2) building blocks for running MPC on OP-TEE Framework

	Coin Tossing Time (s)	Oblivious Transfer Time (s)
Plain	0.028	0.078
Legacy RF	0.048	0.154
`Tr-RF`	0.746	2.561

Table 2: Computation time (in seconds) for Coin Tossing and Oblivious Transfer protocols across Plain, Legacy RF, and `Tr-RF` implementations.

- *Mini-Circuits:* The *mini-circuits* are three distinct components within each *sub-circuit* Γ_i, labeled as $(\Gamma_i^1, \Gamma_i^2, \Gamma_i^3)$. These mini-circuits collectively execute the target functionality using a semi-honest multi-party computation protocol. The master node (M) ensures that each mini-circuit receives uniform inputs and facilitates communication between the mini-circuits within a sub-circuit. In practice, the production of all Γ_i^j can be outsourced to a single polynomially-bounded manufacturer $\mathcal{A}$, which produces physical devices D_i^j intended to implement the mini-circuits' specifications. However, these D_i^j are treated as untrusted hardware components. In our implementation, the mini-circuits are realized as independent applications running within the OP-TEE framework. Leveraging RPI-OP-TEE, these applications inherently support isolation and can only communicate through the master node, ensuring controlled and secure interactions.

Realizing Private Circuit III Assumptions. The choice of basing our implementation on a Trusted Execution Environment is deliberate. **We stress that we do not use the hardware trust assumptions of OP-TEE to derive the guarantees of our Trojan-resilient Reverse Firewall**. We rather use OP-TEE to provide, to the best of our knowledge, the first practical realization of the compiler in [DFS16]. Briefly, an assumption in [DFS16] that forms the foundation of Trojan robustness guarantees is that two mini-circuits do not communicate with each other without the intervention of the master node.

To practically realize this assumption of no communication between mini-circuits, we implement the mini-circuits as Trusted Applications in OP-TEE, and enable remote attestation reports from OP-TEE[1]. Two mini-circuits from [DFS16] can now only communicate with each other if and only if they successfully generate false attestation reports. Formally, by choosing to implement mini-circuits as attested Trusted Applications in OP-TEE, we have reduced the assumptions in [DFS16] to the falsifiable assumption of finding collisions in a collision-resistant hash function. Finding collisions in a collision-resistant hash function is considered to be computationally hard for a probabilistic polynomial time adversary, and is a standard assumption for a wide variety of cryptographic protocols. This reduction of the assumption in [DFS16] to hash-collision builds the core foundation to practically realize our Trojan-resistant Reverse Firewall, and its associated Coin Tossing and OT protocols.

Implementation challenges. While realizing Private Circuit III assumption, we found that OP-TEE, developed by *Linaro*, requires additional patches to work on RPI4 and faces network communication issues. Therefore, we used RPI3 as the standard board for OP-TEE implementation. Additionally, the RPI3-based OP-TEE does not support cryptographic libraries like *openssl* or *mbedtls* in the normal world. As a result, we performed large-number operations exclusively in the secure world, and represented numbers as strings in the normal world.

[1]Remote Attestation involves hardware measurements and application code/data hashed together, and signed using the Original Equipment Manufacturer's private key.

6 Performance Evaluation

We evaluate Trojan-resilient RF for coin tossing and oblivious transfer protocols by comparing it against legacy RF and plain RF implementations with a reverse firewall. The comparison focuses on computation and communication overhead across three coin-tossing implementations: (1) trojan-resilient RF, (2) legacy RF, and (3) plain protocols. Similarly, we analyze three oblivious transfer implementations: (1) trojan-resilient RF, (2) legacy RF, and (3) plain protocols to assess the performance of the proposed approach.

Experimental setup. Our experiment consists of two Raspberry Pi models: (1) RPi4 model B Broadcom BCM2711, Quad core Cortex-A72 (ARM v8) 64-bit SoC @ 1.8GHz equipped with 8GB of RAM, (2) RPi3 Model B v2 Quad Core 1.2GHz Broadcom BCM2837 64-bit CPU equipped with 1 GB RAM. In this setup, we employ a total of 9 RPi3 boards for sub-circuits, each implementing 3 mini-circuits. Each mini-circuit runs as a Trusted Application in the OP-TEE framework. The RPi4 board acts as a trusted master node which performs minimal computation independent of the size of the circuit being evaluated. RPi4 also establishes the socket communication over the LAN network to interact and help mini-circuits communicate among each other.

System and security parameters. We used *mbedtls* library to support big numbers of length 2048-bits. Additionally, we also use AES implementation from *mbedtls* library in counter mode as a pseudo-random generator. We use the same parameter choice as used by *openssl* library, considering the same group with order in the range 2^{2048} as a DDH hard group to implement Pedersen commitment and eventually the coin toss protocol and oblivious transfer protocol. In our experiments, we keep the maximum number of sub-circuits as 9 as it achieves the sufficient level of robustness for trojan detection. Concretely, from [DFS16], given $\lambda = 9$ sub-circuits, which results in 27 mini-circuits (devices) that need to be produced by the manufacturer. If we test each sub-circuit for max $t = 10^9$ runs and use them for $n = 10^5$ executions later, the theorem guarantees that (except with probability 10^{-17}) the resulting computation is correct.

6.1 Baseline Implementations

We evaluate the Trojan-resilient RF for each protocol in comparison to following two baselines:

- **Plain implementation baseline**: The protocol is implemented without a reverse firewall, making it vulnerable to exfiltration if a party's computer is compromised. For coin tossing, we use Blum's protocol [Blu81], and for oblivious transfer, we use Naor-Pinkas/Aiello-Ishai-Raingold [NP01, AIR01] OT protocol.
- **Legacy Reverse Firewall Baseline:** This includes the reverse firewall for the respective protocol. We implement the proposed RF-based coin tossing protocol (Section 4.2) and use the RF-based OT construction from [MSD15] for oblivious transfer.

6.2 Building Blocks

We first establish some building blocks which can be used in any arbitrary protocol for its full-fledged trojan-resilient implementation. In this regard, we explain the implementations of secure addition, secure multiplication, secure public exponentiation, and secure private exponentiation below, and provide their respective computation (see Table 1) and communication overheads. Our secure addition and secure multiplication computations rely on Araki's semi-honest secure three-party computation with an honest majority [AFL$^+$16]. While the secure public exponentiation computation uses techniques from [DFK$^+$06].

- **Secure Addition in OP-TEE Framework.** The master node generates additive shares of x and y (denoted as $[x]_p$ and $[y]_p$) and distributes them to three OP-TEE applications, required for implementing

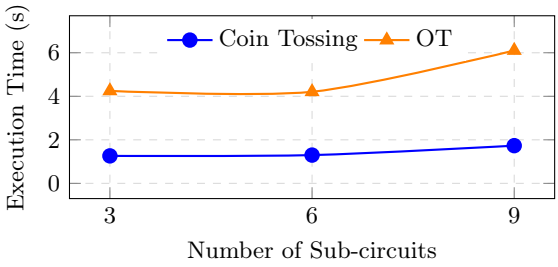

Figure 9: Execution time (in seconds) for coin tossing and oblivious transfer protocols with varying sub-circuits.

the private circuit III compiler. Each application receives the shares in the normal world and forwards them to the secure world. In the secure world, the applications compute $[x+y]_p = [x]_p + [y]_p$, return the result to the normal world, and send $[x+y]_p$ back to the master node. The master node aggregates the shares to reconstruct $x+y$. The communication overhead involves sending two shares ($[x]_p$, $[y]_p$) to each OP-TEE application and receiving one share ($[x+y]_p$), incurring a cost of 3μ per application (where μ is the bit size of each value). With three applications, the total communication cost is 9μ.

- **Secure Multiplication in OP-TEE Framework.** The multiplication operation uses Beaver's triplet to compute on additive shares of operands x and y (see Algorithm 2). The master node generates shares $[x]_p$, $[y]_p$, and Beaver's triples (a, b, c) such that $c = a * b$, along with their respective shares $[a]_p$, $[b]_p$, and $[c]_p$. These shares are distributed to the OP-TEE applications. In the secure world, each application computes $[\delta]_p = [x]_p - [a]_p$ and $[\epsilon]_p = [y]_p - [b]_p$, and the resulting shares are broadcast to other applications through the master node. Each application then computes $[xy]_p = \epsilon[a]_p + \delta[b]_p + [c]_p$. Additionally, one designated application includes the cross-term $\epsilon\delta$. The master node aggregates the final shares from all applications to reconstruct xy. The communication cost includes distributing 5 shares, broadcasting 6 shares, and returning 1 share per application, resulting in 12μ bits per OP-TEE application. With three applications, the total communication cost is 36μ bits.

- **Secure Public Exponentiation in OP-TEE Framework.** For secure public exponentiation, the base x is secret shared, while the exponent y is directly available to all OP-TEE applications. To compute x^y, the master node generates $[x]_p$, samples $r \in \mathbb{Z}_q$, computes r^y, and generates shares $[r]_p$ and $[r^y]_p$. Using secure multiplication, it computes $[xr]_p$ and broadcasts it to all applications. Each OP-TEE application uses the private inverse protocol (Algorithm 4) to compute $[r^{-y}]_p$ from $[r^y]_p$. The applications then calculate $[x^y]_p = (xr)^y[r^{-y}]_p$. Finally, the master node aggregates these shares to reconstruct x^y. A summary of the private inverse protocol is provided later in this section.

  Private Inverse Protocol: Each OP-TEE application holds shares of x and aims to compute shares of x^{-1}. The master node generates a random $r \in \mathbb{Z}_q$, computes its modular inverse, and re-samples if the inverse does not exist. It then shares $[r]_p$ with the OP-TEE applications. The applications perform secure multiplication on shares of r and x to compute shares of xr, broadcast them, and compute $(xr)^{-1}$ if it exists. If $(xr)^{-1}$ does not exist, the protocol is repeated. Each application computes its share of x^{-1} as $[x^{-1}]_p = (xr)^{-1}[r]_p$. The communication overhead includes 10μ for secure multiplication and 3μ for broadcasting, totaling 13μ per application. Across all applications and the master node, the protocol incurs 39μ bits of communication.

  We now analyze the communication overhead of the public exponentiation protocol. The master node shares $[r]_p$ and $[r^y]_p$, then performs secure multiplication to compute shares of $[xr]$, which are broadcasted among OP-TEE applications, requiring 14μ-bits. The private inverse protocol adds 39μ-bits of communication, and sharing the final result incurs an additional μ overhead, requiring 54μ-bits of communication in total.

Performance Evaluation Results. We demonstrate in Figure 9 the execution time of coin tossing protocol and oblivious transfer protocol when the number of sub-circuits is varied from 3 to 9. We note that with a minimal increase in execution time, we achieve Trojan-resilient implementations of these protocols. We

also highlight the trade-off between chance of probabilistic failure vs execution time: Increasing the number of sub-circuits to 9 already gives a failure probability of 10^{-17}, while also giving a realistic computation overhead.

In Table 1, we highlight the execution time for the local operations and building blocks of OP-TEE framework to execute MPC protocols. In Table 2, we provide the details of computation overhead of both coin tossing and Oblivious transfer protocols in three variations: including the plain protocol implementation (with no reverse firewall), Legacy RF based protocol implementation, and Trojan-resilient RF-based protocol implementations. This table provides an insight of security vs efficiency for both the protocols. We however note that we have *implemented* `Tr-RF` through our RPi3 setup, and thus incur network overhead that adds to the communication cost. In reality, we expect `Tr-RF` instantiated with the PC-III compiler to be printed on a *single* Printed Circuit Board (PCB), thereby completely eliminating the network overhead which our implementation currently has. It is an interesting future problem to print our `Tr-RF` implementation on a single PCB and compare the performance of such an implementation with legacy RF [MSD15].

References

[AFL+16] Toshinori Araki, Jun Furukawa, Yehuda Lindell, Ariel Nof, and Kazuma Ohara. High-throughput semi-honest secure three-party computation with an honest majority. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, CCS '16, page 805–817, New York, NY, USA, 2016. Association for Computing Machinery.

[AIR01] Bill Aiello, Yuval Ishai, and Omer Reingold. Priced oblivious transfer: How to sell digital goods. In Birgit Pfitzmann, editor, *Advances in Cryptology — EUROCRYPT 2001*, pages 119–135, Berlin, Heidelberg, 2001. Springer Berlin Heidelberg.

[BBF+20] Angèle Bossuat, Xavier Bultel, Pierre-Alain Fouque, Cristina Onete, and Thyla van Der Merwe. Designing reverse firewalls for the real world. In *Computer Security–ESORICS 2020: 25th European Symposium on Research in Computer Security, ESORICS 2020, Guildford, UK, September 14–18, 2020, Proceedings, Part I 25*, pages 193–213. Springer, 2020.

[Blu81] Manuel Blum. Coin flipping by telephone. In *Advances in Cryptology: A Report on CRYPTO 81*, pages 11–15, 1981.

[CDN20] Suvradip Chakraborty, Stefan Dziembowski, and Jesper Buus Nielsen. Reverse firewalls for actively secure mpcs. In *Annual international cryptology conference*, pages 732–762. Springer, 2020.

[CGPS21] Suvradip Chakraborty, Chaya Ganesh, Mahak Pancholi, and Pratik Sarkar. Reverse firewalls for adaptively secure mpc without setup. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 335–364. Springer, 2021.

[CGS23] Suvradip Chakraborty, Chaya Ganesh, and Pratik Sarkar. Reverse firewalls for oblivious transfer extension and applications to zero-knowledge. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 239–270. Springer, 2023.

[CMMV25] Suvradip Chakraborty, Lorenzo Magliocco, Bernardo Magri, and Daniele Venturi. Key exchange in the post-snowden era: Universally composable subversion-resilient pake. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 101–133. Springer, 2025.

[CMNV22] Suvradip Chakraborty, Bernardo Magri, Jesper Buus Nielsen, and Daniele Venturi. Universally composable subversion-resilient cryptography. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 272–302. Springer, 2022.

[DFK+06] Ivan Damgård, Matthias Fitzi, Eike Kiltz, Jesper Buus Nielsen, and Tomas Toft. Unconditionally secure constant-rounds multi-party computation for equality, comparison, bits and exponentiation. In Shai Halevi and Tal Rabin, editors, *Theory of Cryptography*, pages 285–304, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg.

[DFS16] Stefan Dziembowski, Sebastian Faust, and François-Xavier Standaert. Private circuits iii: Hardware trojan-resilience via testing amplification. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, CCS '16, page 142–153, New York, NY, USA, 2016. Association for Computing Machinery.

[DMSD16] Yevgeniy Dodis, Ilya Mironov, and Noah Stephens-Davidowitz. Message transmission with reverse firewalls–secure communication on corrupted machines. In *Proceedings, Part I, of the 36th Annual International Cryptology Conference on Advances in Cryptology — CRYPTO 2016 - Volume 9814*, page 341–372, Berlin, Heidelberg, 2016. Springer-Verlag.

[Gro09] Jens Groth. Homomorphic trapdoor commitments to group elements. *IACR Cryptol. ePrint Arch.*, 2009:7, 2009.

[IEGT13] Frank Imeson, Ariq Emtenan, Siddharth Garg, and Mahesh V. Tripunitara. Securing computer hardware using 3d integrated circuit (ic) technology and split manufacturing for obfuscation. In *Proceedings of the 22nd USENIX Conference on Security*, SEC'13, page 495–510, USA, 2013. USENIX Association.

[JSS20] Patrick Jauernig, Ahmad-Reza Sadeghi, and Emmanuel Stapf. Trusted execution environments: Properties, applications, and challenges. *IEEE Security & Privacy*, 18(2):56–60, 2020.

[Lin17] Yehuda Lindell. *How to Simulate It – A Tutorial on the Simulation Proof Technique*, pages 277–346. Springer International Publishing, Cham, 2017.

[MSD15] Ilya Mironov and Noah Stephens-Davidowitz. Cryptographic reverse firewalls. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology - EUROCRYPT 2015*, pages 657–686, Berlin, Heidelberg, 2015. Springer Berlin Heidelberg.

[NP01] Moni Naor and Benny Pinkas. Efficient oblivious transfer protocols. In *Proceedings of the Twelfth Annual ACM-SIAM Symposium on Discrete Algorithms*, SODA '01, page 448–457, USA, 2001. Society for Industrial and Applied Mathematics.

[Ped91] Torben Pryds Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *Annual international cryptology conference*, pages 129–140. Springer, 1991.

[WS10] Adam Waksman and Simha Sethumadhavan. Tamper evident microprocessors. In *2010 IEEE Symposium on Security and Privacy*, pages 173–188, 2010.

[WS11] Adam Waksman and Simha Sethumadhavan. Silencing Hardware Backdoors . In *2012 IEEE Symposium on Security and Privacy*, pages 49–63, Los Alamitos, CA, USA, May 2011. IEEE Computer Society.

[WTP08] Xiaoxiao Wang, Mohammad Tehranipoor, and Jim Plusquellic. Detecting malicious inclusions in secure hardware: Challenges and solutions. In *2008 IEEE International Workshop on Hardware-Oriented Security and Trust*, pages 15–19, 2008.

Appendix

Algorithm 5 Conditional Multiplication Protocol

Require: share $[x]_p$, share $[r]_p$, $[\mathsf{bitShare}]_p$ and modulus q.
Ensure: either share $[xr]_p$ or share $[x]_p$.
1: $[\mathsf{One}]_p \leftarrow$ SECRET.SHARE(1)
2: $[s]_p = [r]_p - [\mathsf{One}]_p$
3: $[y]_p =$ MULTIPLY$([\mathsf{bitShare}]_p, [s]_p, q)$
4: $[y]_p = [y]_p - [\mathsf{One}]_p$
5: $[x]_p =$ MULTIPLY$([x]_p, [y]_p, q)$
6: **return** $[x]_p$

Algorithm 6 Private Exponentiation Protocol

Require: base share $[x]_p$, list of binary exponent share $[y^0]_p, [y^1]_p, \ldots, [y^(l-l)]_p$ and modulus q.
Ensure: share $[x^y]_p$.
1: $[\mathsf{res}]_p \leftarrow$ SECRET.SHARE(1)
2: **for** $[\mathsf{bitShare}]_p$ in $[[y^(l-l)]_p, [y^(l-2)]_p, \ldots, [y^0]_p]$ **do**
3: $\quad [r]_p = [x]_p$
4: $\quad [\mathsf{res}]_p =$ CONDITIONALMULTIPLY$([\mathsf{res}]_p, [r]_p, [\mathsf{bitShare}]_p, q)$
5: $\quad [x]_p =$ MULTIPLY$([x]_p, [x]_p, q)$
6: **return** $[\mathsf{res}]_p$

A An Alternate Design Model

In this section, we discuss an alternate design model based on a different assumptions. In this model, the master node is not probably required. In this model, the master node does not generate the randomness of coin tossing and oblivious transfer protocol. However, we just require one of the three parties to be honest and our design model can provide guarantee of implementation `Tr-RF`. This model require the private exponentiation protocol as describe below.

Secure Private Exponentiation in the OP-TEE Framework. In this protocol, each application holds shares of x as $[x]_p$ and shares of y as $[y]_p$, aiming to compute x^y. The master node selects a random value r and generates its additive shares $[r]_p$ along with bit-wise additive shares. For $r = \sum_{i=0}^{\ell-1} r_i 2^i$, the bit-wise additive shares are represented as $([r_0]_p, \ldots, [r_{\ell-1}]_p)$. These shares are distributed to the respective OP-TEE applications.

The applications collaboratively run a secure addition protocol over $[y]_p$ and $[r]_p$, broadcasting the result $c = y + r$. They then invoke a secure public exponentiation protocol to compute shares of x^c as $[x^c]_p$. Each OP-TEE application maintains storage for 2^i values for all $i \in \ell$ and computes shares of $x^{2^{\ell-i-1}}$ through the public exponentiation protocol.

Next, they execute a secure conditional multiplication protocol, starting by computing shares of $x^{2^{\ell-i-1}} - 1$ for each $i \in l$. They then perform secure multiplications using the bit shares and shares of $x^{2^{\ell-i-1}} - 1$, followed by secure additions with shares of 1, followed by secure multiplication with shares of from last step and shares of 1. This step ensures that if the bit position is 1, the result is a share of $x^{2^{\ell-i-1}}$; otherwise, it remains a share of 1.

The final result of the exponentiation is computed through secure multiplications over all bit positions followed by another secure multiplication with inverse shares of r (They need to run private inverse protocol to get shares of r^{-1}). The complete protocol requires ℓ secure public exponentiations and $\ell - 1$ secure multiplications. Each bit-wise conditional multiplication involves 2 secure additions and 2 secure multiplications.

In total, the protocol involves $\ell + 1$ exponentiations, 3ℓ secure multiplications, and $2l$ secure additions. The private inverse protocol takes 39μ-bits to communicate over the channel. Considering communication overhead, the total complexity amounts to $54(l + 1)\mu + 36(3\ell)\mu + 9(2\ell)\mu + 39\mu = (180l + 93)\mu$.